

4.5.1 IPv6 的基本首部

IPv6 仍支持无连接的传送, 但将协议数据单元 PDU 称为分组(packet), 而不是 IPv4 的数据报(datagram)。为方便起见, 本书仍采用数据报这一名词 ([KURO17]和[TANE11]也是这样做的)。实际上, 在本书中一直把分组和数据报看成是同义词。

IPv6 所引进的主要变化如下:

- (1) **更大的地址空间。**IPv6 把地址从 IPv4 的 32 位增大到 4 倍, 即增大到 128 位, 使地址空间增大了 2^96 倍。这样大的地址空间在可预见的将来是不会用完的。
- (2) **扩展的地址层次结构。**IPv6 由于地址空间很大, 因此可以划分为更多的层次。
- (3) **灵活的首部格式。**IPv6 数据报的首部和 IPv4 的并不兼容。IPv6 定义了许多可选的扩展首部, 不仅可提供比 IPv4 更多的功能, 而且还可提高路由器的处理效率, 这是因为路由器对扩展首部不进行处理 (除逐跳扩展首部外)。
- (4) **改进的选项。**IPv6 允许数据报包含有选项的控制信息, 因而可以包含一些新的选项。但 IPv6 的首部长度是固定的, 其选项放在有效载荷中。我们知道, IPv4 所规定的选项是固定不变的, 其选项放在首部的可变部分。
- (5) **允许协议继续扩充。**这一点很重要, 因为技术总是在不断地发展的 (如网络硬件的更新), 而新的应用也还会出现。但我们知道, IPv4 的功能是固定不变的。
- (6) **支持即插即用 (即自动配置)。**因此 IPv6 不需要使用 DHCP。
- (7) **支持资源的预分配。**IPv6 支持实时视像等要求保证一定的带宽和时延的应用。
- (8) **IPv6 首部改为 8 字节对齐 (即首部长度必须是 8 字节的整数倍)。**原来的 IPv4 首部是 4 字节对齐。

IPv6 数据报由两大部分组成, 即**基本首部(base header)**和后面的**有效载荷(payload)**。有效载荷也称为**净负荷**。有效载荷允许有零个或多个**扩展首部(extension header)**, 再后面是数据部分 (如图 4-32 所示)。但请注意, 所有的扩展首部并不属于 IPv6 数据报的基本首部。

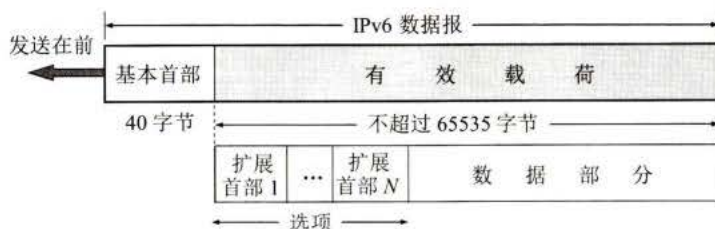


图 4-32 具有多个可选扩展首部的 IPv6 数据报的一般形式

与 IPv4 相比, IPv6 对首部中的某些字段进行了如下的更改:

- 取消了首部长度的字段, 因为它的首部长度是固定的 (40 字节)。
- 取消了服务类型字段, 因为优先级和流标号字段实现了服务类型字段的功能。
- 取消了总长度字段, 改用有效载荷长度字段。
- 取消了标识、标志和片偏移字段, 因为这些功能已包含在分片扩展首部中。
- 把 TTL 字段改称为跳数限制字段, 但作用是一样的 (名称与作用更加一致)。
- 取消了协议字段, 改用下一个首部字段。
- 取消了检验和字段, 这样就加快了路由器处理数据报的速度。我们知道, 在数据链

路层对检测出有差错的帧就丢弃。在运输层，当使用 UDP 时，若检测出有差错的 用户数据报就丢弃。当使用 TCP 时，对检测出有差错的报文段就重传，直到正确 传送到目的进程为止。因此在网络层的差错检测可以精简掉。

- 取消了选项字段，而用扩展首部来实现选项功能。

由于把首部中不必要的功能取消了，使得 IPv6 首部的字段数减少到只有 8 个（虽然首 部长度增大了一倍）。

下面解释 IPv6 基本首部中各字段的作用（参见图 4-33）。

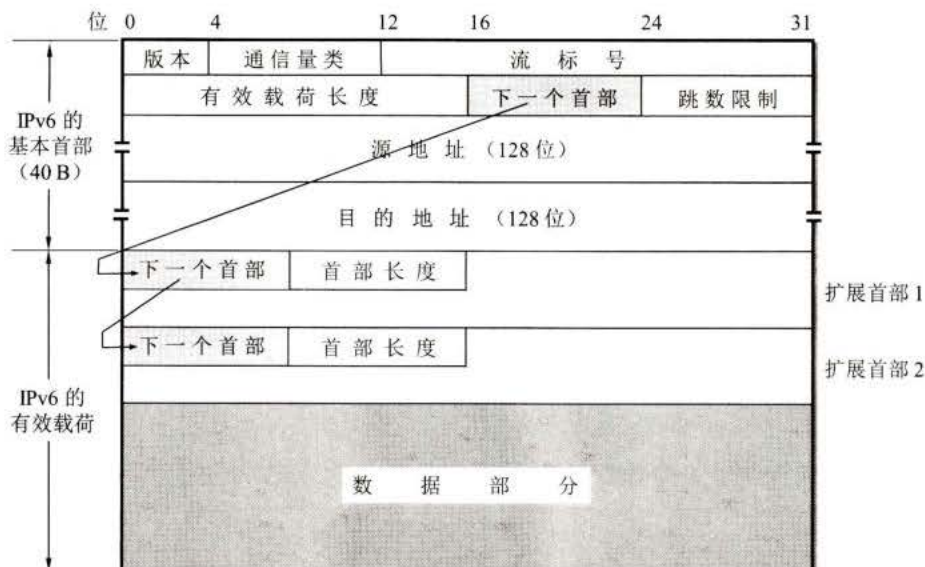


图 4-33 IPv6 基本首部和有效载荷（这里画出了两个扩展首部作为例子）

- (1) 版本(version) 占 4 位。它指明了协议的版本，对 IPv6 该字段是 6。
- (2) 通信量类(traffic class) 占 8 位。这是为了区分不同的 IPv6 数据报的类别或优先级，和 IPv4 的区分服务字段的作用相似。目前正在进行不同的通信量类性能的实验。
- (3) 流标号(flow label) 占 20 位。IPv6 的一个新的机制是支持资源预分配，并且允许路由器把每一个数据报与一个给定的资源分配相联系。IPv6 提出流(flow)的抽象概念。所谓“流”就是互网络上从特定源点到特定终点（单播或多播）的一系列数据报（如实时音频或视频传输），而在这个“流”所经过的路径上的路由器都保证指明的服务质量。所有属于同一个流的数据报都具有同样的流标号。因此，流标号对实时音频/视频数据的传送特别有用。对于传统的电子邮件或非实时数据，流标号则没有用处，把它置为 0 即可。关于流标号的规约可参考建议标准[RFC 6437]。
- (4) 有效载荷长度(payload length) 占 16 位。它指明 IPv6 数据报除基本首部以外的字节数（所有扩展首部都算在有效载荷之内）。这个字段的最大值是 64 KB（65535 字节）。
- (5) 下一个首部(next header) 占 8 位。它相当于 IPv4 的协议字段或可选字段。
 - 当 IPv6 数据报没有扩展首部时，下一个首部字段的作用和 IPv4 的协议字段一样，它的值指出了基本首部后面的数据应交付 IP 层上面的哪一个高层协议（例如：6 或 17 分别表示应交付运输层 TCP 或 UDP）。
 - 当出现扩展首部时，下一个首部字段的值就标识后面第一个扩展首部的类型。

(6) 跳数限制(hop limit) 占 8 位。用来防止数据报在网络中无限期地存在。和 IPv4 的生存时间字段相似。源点在每个数据报发出时即设定某个跳数限制(最大为 255 跳)。每个路由器在转发数据报时,要先把跳数限制字段中的值减 1。当跳数限制的值为零时,就要把这个数据报丢弃。

(7) 源地址 占 128 位。是数据报的发送端的 IP 地址。

(8) 目的地址 占 128 位。是数据报的接收端的 IP 地址。

下面我们简单介绍一下 IPv6 的扩展首部。

在 RFC 8200 中定义了以下六种扩展首部: (1) 逐跳选项; (2) 路由选择; (3) 分片; (4) 鉴别; (5) 封装安全有效载荷; (6) 目的站选项。

每一个扩展首部都由若干个字段组成,它们的长度也各不同。但所有扩展首部的第一个字段都是 8 位的“下一个首部”字段。此字段的值指出了在该扩展首部后面的扩展首部是什么。当使用多个扩展首部时,应按以上的先后顺序出现。高层首部总是放在最后面。

大家知道,IPv4 的数据报若在其首部中使用了选项,则在数据报转发路径中的每一个路由器,都必须检查首部中的所有选项,看是否与本路由器相关。这必然要花费相当的时间。IPv6 把原来 IPv4 首部中选项的功能都放在扩展首部中。IPv6 数据报若使用了扩展首部,则其基本首部的“下一个首部”字段会指出,在“有效载荷”字段中使用了何种扩展首部。而所有扩展首部的第一个字段都是“下一个首部”,用来指出在后面还有何种扩展首部。这就使得路由器能够迅速判断待转发的 IPv6 数据报有无需要本路由器处理的选项。

4.5.2 IPv6 的地址

一般来讲,一个 IPv6 数据报的目的地址可以是以下三种基本类型地址之一:

(1) 单播(unicast) 单播就是传统的点对点通信。

(2) 多播(multicast) 多播是一点对多点的通信,数据报发送到一组计算机中的每一个。IPv6 没有采用广播的术语,而是将广播看作多播的一个特例。

(3) 任播(anycast) 这是 IPv6 增加的一种类型。任播的终点是一组计算机,但数据报只交付其中的一个,通常是按照路由算法得出的距离最近的一个。

IPv6 把实现 IPv6 的主机和路由器均称为节点。由于一个节点可能会使用多条链路与其他的一些节点相连,因此一个节点可能有多个与链路相连的接口。这样,IPv6 给节点的每一个接口(请注意,不是给某个节点)指派一个 IPv6 地址。一个具有多个接口的节点可以有多个单播地址,而其中任何一个地址都可当作到达该节点的目的地址。不过有时为了方便,若不会引起误解,也常说某个节点的 IPv6 地址,而把某个接口省略掉。

在 IPv6 中,每个地址占 128 位,地址空间大于 3.4×10^{38} 。如果整个地球表面(包括陆地和水面)都覆盖着计算机,那么 IPv6 允许每平方米拥有 7×10^{23} 个 IP 地址。如果地址分配速率是每微秒分配 100 万个地址,则需要 10^{19} 年的时间才能将所有可能的地址分配完毕。可见在想象到的将来,IPv6 的地址空间是不可能用完的。

为了体会一下 IPv6 的地址有多大,可以看一下目前已经分配出去的最大的地址块。法国电信 France Telecom 和德国电信 Deutsche Telekom 各分配到一个 /19 地址块,相当于各有 35×10^{12} 个地址,远远大于全部的 IPv4 地址(IPv4 地址还不到 4.3×10^9 个)。

巨大的地址范围还必须使维护互联网的人易于阅读和操纵这些地址。IPv4 所用的点分十进制记法现在也不够方便了。例如,一个用点分十进制记法的 128 位的地址为:

扫一扫



视频讲解

104.230.140.100.255.255.255.255.0.0.17.128.150.10.255.255

为了使地址再稍简洁些, IPv6 使用**冒号十六进制记法**(colon hexadecimal notation, 简称为 colon hex), 它把每个 16 位的值用十六进制值表示, 各值之间用冒号分隔。例如, 如果前面所给的点分十进制数记法的值改为冒号十六进制记法, 就变成了:

68E6:8C64:FFFF:FFFF:0:1180:960A:FFFF

在十六进制记法中, 允许把数字前面的 0 省略。上面就把 0000 中的前三个 0 省略了。

冒号十六进制记法还包含两个技术使它尤其有用。首先, 冒号十六进制记法可以允许**零压缩**(zero compression), 即一连串连续的零可以为一对冒号所取代, 例如:

FF05:0:0:0:0:0:0:B3

可压缩为:

FF05::B3

为了保证零压缩有一个不含混的解释, 规定在任一地址中只能使用一次零压缩。该技术对已建议的分配策略特别有用, 因为会有许多地址包含较长连续的零串。

其次, 冒号十六进制记法可结合使用点分十进制记法的后缀。我们下面会看到这种结合在 IPv4 向 IPv6 的转换阶段特别有用。例如, 下面的串是一个合法的冒号十六进制记法:

0:0:0:0:0:0:128.10.2.1

请注意, 在这种记法中, 冒号所分隔的每个值是两个字节(16 位)的值, 但点分十进制每个部分的值是一个字节(8 位)的值。再使用零压缩即可得出:

::128.10.2.1

下面再给出几个使用零压缩的例子。

1080:0:0:0:8:800:200C:417A 记为 1080::8:800:200C:417A

FF01:0:0:0:0:0:0:101 (多播地址) 记为 FF01::101

0:0:0:0:0:0:0:1 (环回地址) 记为 ::1

0:0:0:0:0:0:0:0 (未指明地址) 记为 ::

CIDR 的斜线表示法仍然可用。例如, 60 位的前缀 12AB00000000CD3 (十六进制表示的 15 个字符, 每个字符代表 4 位二进制数字) 可记为:

12AB:0000:0000:CD30:0000:0000:0000:0000/60

或 12AB::CD30:0:0:0:0/60

或 12AB:0:0:CD30::/60

但不允许记为:

12AB:0:0:CD3/60 (不能把 16 位地址 CD30 块中的最后的 0 省略)

或 12AB::CD30/60 (这表示 12AB:0:0:0:0:0:0:CD30/60)

或 12AB::CD3/60 (这表示 12AB:0:0:0:0:0:0:0CD3/60)

但是, IPv6 取消了子网掩码。

斜线的意思和 IPv4 的情况相似。例如,

CIDR 记法的 2001:0DB8:0:CD30:123:4567:89AB:CDEF/60，表示 IPv6 的地址是：2001:0DB8:0:CD30:123:4567:89AB:CDEF 而其子网号是：2001:0DB8:0:CD30::/60 IPv6 的地址分类如表 4-7 所示[RFC 4291]。

表 4-7 IPv6 的常用地址分类

地址类型	地址块前缀	前缀的 CIDR 记法
未指明地址	00...0 (128 位)	::/128
环回地址	00...1 (128 位)	::1/128
多播地址	11111111	FF00::/8
本地站点单播地址	1111111011	FEC0::/10
本地链路单播地址	1111111010	FE80::/10
全球单播地址	见图 4-34	

对表 4-7 所列举的几种常用地址简单解释如下。

未指明地址 这是 16 字节的全 0 地址，可缩写为两个冒号“::”。这个地址不能用作目的地址，而只能将某台主机当作源地址使用，条件是这台主机还没有配置到一个标准的 IP 地址。这类地址仅此一个。

环回地址 IPv6 的环回地址是 0:0:0:0:0:0:0:1，可缩写为::1。它的作用和 IPv4 的环回地址一样。这类地址也是仅此一个。

多播地址 功能和 IPv4 的一样。这类地址占 IPv6 地址总数的 1/256。

本地站点单播地址(cite-local unicast address) 有些单位的内部网络使用 TCP/IP 协议，但并没有连接到互联网上。连接在这样的内部网络上的主机都可以使用这种本地站点地址进行通信，但不能和互联网上的其他主机通信。这类地址占 IPv6 地址总数的 1/1024，其用途和和 IPv4 的专用地址是一样的。

本地链路单播地址(link-local unicast address) 这种地址是在单一链路上使用的。当一个节点启用 IPv6 时就自动生成本地链路地址（请注意，这个节点现在并没有连接在某个网络上）。当需要把分组发往单一链路的设备而不希望该分组被转发到此链路范围以外的地方时，就可以使用这种特殊地址。这类地址占 IPv6 地址总数的 1/1024。

全球单播地址 IPv6 的这一类单播地址是使用得最多的一类。曾提出过多种方案来进一步划分这 128 位的单播地址。根据 2006 年发布的草案标准 RFC 4291 的建议，IPv6 单播地址的划分方法非常灵活，可以是如图 4-34 所示的任何一种。这就是说，可把整个的 128 位都作为一个节点的地址。也可用 n 位作为子网前缀，用剩下的 $(128 - n)$ 位作为接口标识符（相当于 IPv4 的主机号）。当然也可以划分为三级，用 n 位作为全球路由选择前缀，用 m 位作为子网前缀，而用剩下的 $(128 - n - m)$ 位作为接口标识符。

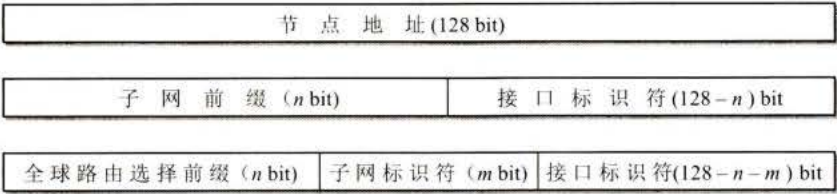


图 4-34 IPv6 的单播地址的几种划分方法

4.5.3 从 IPv4 向 IPv6 过渡

扫一扫



视频讲解

由于现在整个互联网的规模太大，因此，“规定一个日期，从这一天起所有的路由器一律都改用 IPv6”，显然是不可行的。这样，向 IPv6 过渡只能采用逐步演进的办法，同时，还必须使新安装的 IPv6 系统能够向后兼容。

这就是说，IPv6 系统必须能够接收和转发 IPv4 分组，并且能够为 IPv4 分组选择路由。

下面介绍两种向 IPv6 过渡的策略，即使用双协议栈和使用隧道技术[RFC 2473, 2529, 3056, 4038, 4213]。

1. 双协议栈

双协议栈(dual stack)是指在完全过渡到 IPv6 之前，使一部分主机（或路由器）同时装有 IPv4 和 IPv6 这两种协议栈。因此双协议栈主机（或路由器）既能够和 IPv6 的系统通信，又能够和 IPv4 的系统通信。双协议栈的主机（或路由器）记为 IPv6/IPv4，表明它同时具有 IPv6 地址和 IPv4 地址（如图 4-35 所示）。



图 4-35 使用双协议栈进行从 IPv4 到 IPv6 的过渡

双协议栈的主机在和 IPv6 主机通信时采用 IPv6 地址，而和 IPv4 主机通信时则采用 IPv4 地址。但双协议栈主机怎样知道目的主机是采用哪一种地址呢？它是使用域名系统 DNS 来查询的。若 DNS 返回的是 IPv4 地址，则双协议栈的源主机就使用 IPv4 地址。但当 DNS 返回的是 IPv6 地址，源主机就使用 IPv6 地址。

双协议栈需要付出的代价太大，因为要安装上两套协议。因此在过渡时期，最好采用下面的隧道技术。

2. 隧道技术

向 IPv6 过渡的另一种方法是隧道技术(tunneling)。图 4-36 给出了隧道技术的工作原理。这种方法的要点就是在 IPv6 数据报要进入 IPv4 网络时，把 IPv6 数据报封装成为 IPv4 数据报。现在整个的 IPv6 数据报变成了 IPv4 数据报的数据部分。这样的 IPv4 数据报从路由器 B 经过路由器 C 和 D，传送到 E，而原来的 IPv6 数据报就好像在 IPv4 网络的隧道中传输，什么都没有变化。当 IPv4 数据报离开 IPv4 网络中的隧道时，再把数据部分（即原来的 IPv6 数据报）交给主机的 IPv6 协议栈。图中的一条粗线表示在 IPv4 网络中好像有一个从 B 到 E 的“IPv6 隧道”，路由器 B 是隧道的入口而 E 是出口。请注意，在隧道中传送的数据报的源地址是 B 而目的地址是 E。

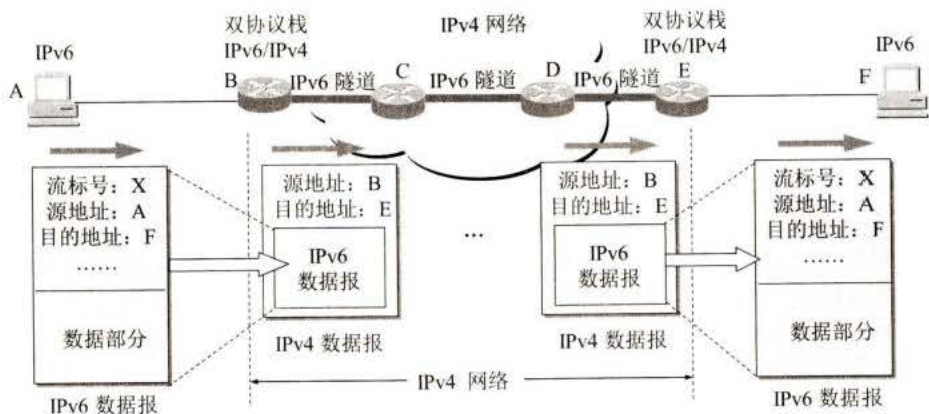


图 4-36 使用隧道技术进行从 IPv4 到 IPv6 的过渡

要使双协议栈的主机知道 IPv4 数据报里面封装的数据是一个 IPv6 数据报，就必须把 IPv4 首部的协议字段的值设置为 41（41 表示数据报的数据部分是 IPv6 数据报）。

4.5.4 ICMPv6

和 IPv4 一样，IPv6 也不保证数据报的可靠交付，因为互联网中的路由器可能会丢弃数据报。因此 IPv6 也需要使用 ICMP 来反馈一些差错信息。新的版本称为 ICMPv6，它比 ICMPv4 要复杂得多。地址解析协议 ARP 和网际组管理协议 IGMP 的功能都已被合并到 ICMPv6 中（如图 4-37 所示）。



图 4-37 新旧版本中的网络层的比较

ICMPv6 是面向报文的协议，它利用报文来报告差错，获取信息，探测邻站或管理多播通信。ICMPv6 还增加了几个定义报文功能及含义的其他协议。在对 ICMPv6 报文进行归类时，不同的文献和 RFC 文档使用了不同的策略，有的把其中的一些报文定义为 ICMPv6 报文，而把另一些报文定义为邻站发现 ND(Neighbor-Discovery)报文或多播听众交付 MLD (Multicast Listener Delivery)报文。其实所有这些报文都应当是 ICMPv6 报文，只是功能和作用不同而已。因此我们把这些报文都列入 ICMPv6 的不同类别。使用这种分类方法的原因是所有这些报文都具有相同的格式，并且所有报文类型都由 ICMPv6 协议处理。其实，像 ND 和 MLD 这样的协议都是运行在 ICMPv6 协议之下的。基于这样的考虑，可把 ICMPv6 报文分类，如图 4-38 所示。请注意，邻站发现报文和组成员关系报文分别是在 ND 协议和 MLD 协议的控制下进行发送和接收的。

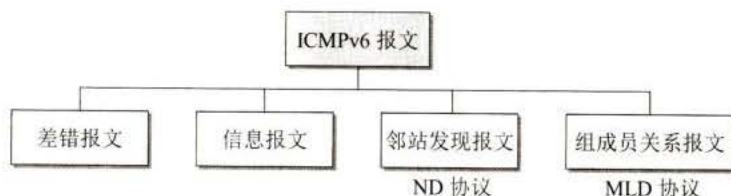


图 4-38 ICMPv6 报文的分类

关于 ICMPv6 的进一步讨论可参阅[FORO10]，这里从略。

4.6 互联网的路由选择协议

本节将讨论几种常用的路由选择协议，也就是要讨论转发表中的路由是怎样得出的。按照 4.1.2 节所述的观点，路由选择协议属于网络层控制层面的内容。不过本节仍然按传统的思路进行讨论，也就是说，路由选择协议规定了互联网中有关的路由器应如何相互交换信息并生成出路由表。

4.6.1 有关路由选择协议的几个基本概念

1. 理想的路由算法

路由选择协议的核心就是路由算法，即需要何种算法来获得路由表中的各项目。一个理想的路由算法应具有如下的一些特点[BELL86]：

(1) **算法必须是正确的和完整的。**这里，“正确”的含义是：沿着各路由表所指引的路由，分组一定能够最终到达目的网络和目的主机。

(2) **算法在计算上应简单。**路由选择的计算不应使网络通信量增加太多的额外开销。

(3) **算法应能适应通信量和网络拓扑的变化，这就是说，要有自适应性。**当网络中的通信量发生变化时，算法能自适应地改变路由以均衡各链路的负载。当某个或某些节点、链路发生故障不能工作，或者修理好了再投入运行时，算法也能及时地改变路由。有时称这种自适应性为“稳健性”(robustness)。^①

(4) **算法应具有稳定性。**在网络通信量和网络拓扑相对稳定的情况下，路由算法应收敛于一个可以接受的解，而不应使得出的路由不停地变化。

(5) **算法应是公平的。**路由选择算法应对所有用户（除对少数优先级高的用户）都是平等的。例如，若仅仅使某一对用户的端到端时延为最小，却而不考虑其他的广大用户，这就明显地不符合公平性的要求。

(6) **算法应是最佳的。**路由选择算法应当能够找出最好的路由，使得分组平均时延最小而网络的吞吐量最大。虽然我们希望能得到“最佳”的算法，但这并不总是最重要的。对于某些网络，网络的可靠性有时要比最小的分组平均时延或最大吞吐量更加重要。因此，所谓“最佳”只能是相对于某一种特定要求下得出的较为合理的选择而已。

一个实际的路由选择算法，应尽可能接近于理想的算法。在不同的应用条件下，对以

^① 注：robustness 一词在自动控制界的标准译名是“鲁棒性”，但在[MINGCI94]则译为“稳健性”。

上提出的 6 个方面也可有不同的侧重。

应当指出, 路由选择是个非常复杂的问题, 因为它是网络中的所有节点共同协调工作的结果。其次, 路由选择的环境往往是不断变化的, 而这种变化有时无法事先知道, 例如, 网络中出了某些故障。此外, 当网络发生拥塞时, 就特别需要有能缓解这种拥塞的路由选择策略, 但恰好在这种条件下, 很难从网络中的各节点获得所需的路由选择信息。

倘若从路由算法能否随网络的通信量或拓扑自适应地进行调整变化来划分, 则只有两大类, 即**静态路由选择策略**与**动态路由选择策略**。静态路由选择也叫作**非自适应路由选择**, 其特点是简单和开销较小, 但不能及时适应网络状态的变化。对于很简单的小网络, 完全可以采用静态路由选择, 用人工配置每一条路由。动态路由选择也叫作**自适应路由选择**, 其特点是能较好地适应网络状态的变化, 但实现起来较为复杂, 开销也比较大。因此, 动态路由选择适用于较复杂的大网络。

2. 分层次的路由选择协议

互联网采用的路由选择协议主要是自适应的(即动态的)、分布式路由选择协议。由于以下两个原因, 互联网采用分层次的路由选择协议:

(1) 互联网的规模非常大。如果让所有的路由器知道所有的网络应怎样到达, 则这种路由表将非常大, 处理起来也太花时间。而所有这些路由器之间交换路由信息所需的带宽就会使互联网的通信链路饱和。

(2) 许多单位不愿意外界了解自己单位网络的布局细节和本部门所采用的路由选择协议(这属于本部门内部的事情), 但同时还希望连接到互联网上。

为此, 可以把整个互联网划分为许多较小的**自治系统**(autonomous system), 一般都记为 AS。自治系统 AS 是在单一技术管理下的许多网络、IP 地址以及路由器, 而这些路由器使用一种自治系统内部的路由选择协议和共同的度量。每一个 AS 对其他 AS 表现出的是一个**单一的和一致的路由选择策略**[RFC 4271]。这样, 互联网就把路由选择协议划分为两大类, 即:

(1) **内部网关协议 IGP** (Interior Gateway Protocol) 即在一个自治系统内部使用的路由选择协议, 而这与在互联网中的其他自治系统选用什么路由选择协议无关。目前这类路由选择协议使用得最多的是 RIP 和 OSPF 协议 (IS-IS 协议也很流行, 但不介绍了)。

(2) **外部网关协议 EGP** (External Gateway Protocol) 若源主机和目的主机处在不同的自治系统中(这两个自治系统可能使用不同的内部网关协议), 那么在不同自治系统 AS 之间的路由选择, 就需要使用外部网关协议 EGP。目前使用最多的外部网关协议是 BGP 的版本 4 (BGP-4)。

自治系统之间的路由选择也叫作**域间路由选择**(interdomain routing), 而在自治系统内部的路由选择叫作**域内路由选择**(intradomain routing)。

图 4-39 是两个自治系统互连在一起的示意图。每个自治系统自己决定在本自治系统内部运行哪一个内部路由选择协议(例如, 可以是 RIP, 也可以是 OSPF)。但每个自治系统都有一个或多个路由器(图中的路由器 R_1 和 R_2)除运行本系统的内部路由选择协议外, 还要运行自治系统间的路由选择协议 (BGP-4)。

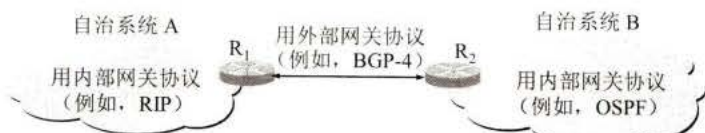


图 4-39 自治系统和内部网关协议、外部网关协议

这里我们要指出两点：

(1) 互联网的早期 RFC 文档中未使用“路由器”而是使用“网关”这一名词。但是在后来的 RFC 文档中又改用“路由器”这一名词，因此有的书把原来的 IGP 和 EGP 分别改为 IRP（内部路由器协议）和 ERP（外部路由器协议）。为了方便读者查阅 RFC 文档，本书仍使用 RFC 原先使用的名字 IGP 和 EGP。

(2) RFC 采用的名词 IGP 和 EGP 是协议类别的名称。但 RFC 在使用名词 EGP 时出现了一点混乱，因为最早的一个外部网关协议的协议名字正好也是 EGP [RFC 827]。后来发现该 RFC 提出的 EGP 有不少缺点，就设计了一种更好的外部网关协议，叫作边界网关协议 BGP (Border Gateway Protocol)，用来取代旧的 RFC 827 外部网关协议 EGP。实际上，旧协议 EGP 和新协议 BGP 都属于外部网关协议 EGP 这一类别。因此在遇到名词 EGP 时，应弄清它是指旧协议 EGP（即 RFC 827）还是指外部网关协议 EGP 这个类别。

总之，使用分层次的路由选择方法，可将互联网的路由选择协议划分为：

- 内部网关协议 IGP：具体的协议有多种，如 RIP 和 OSPF 等。
- 外部网关协议 EGP：目前使用的协议是 BGP-4。

对于比较大的自治系统，还可将所有的网络再进行一次划分。例如，可以构筑一个链路速率较高的主干网和许多速率较低的区域网。每个区域网通过路由器连接到主干网。当在一个区域内找不到目的站时，就通过路由器经过主干网到达另一个区域网，或者通过边界路由器到别的自治系统中去查找。下面对这两类协议分别进行介绍。

4.6.2 内部网关协议 RIP

1. 协议 RIP 的工作原理

RIP (Routing Information Protocol) 是内部网关协议 IGP 中最先得到广泛使用的协议 [RFC 1058]，它的中文译名是路由信息协议。RIP 是一种分布式的基于距离向量的路由选择协议，是互联网的标准协议，其最大优点就是简单。

RIP 协议要求网络中的每一个路由器都要维护从它自己到其他每一个目的网络的距离记录（因此，这是一组距离，即“距离向量”）。协议 RIP 将“距离”定义如下：

从一路由器到直接连接的网络的距离定义为 1。从一主机到非直接连接的网络的距离定义为所经过的路由器数加 1。“加 1”是因为到达目的网络后就进行直接交付（不需要再经过路由器），而到直接连接的网络的距离已经定义为 1。

协议 RIP 的“距离”^①也称为“跳数” (hop count)，并且每经过一个网络，跳数就加 1。RIP 认为好的路由就是它通过的网络数目少，即“距离短”。RIP 允许一条路径最多只能包含 15 个网络。因此“距离”等于 16 时即相当于不可达。可见 RIP 只适用于小型互联网。

^① 注：这里的“距离”实际上指的是“最短距离”，但为方便起见往往省略“最短”二字。

扫一扫



视频讲解

例如在前面的图 4-7 中, 主机 H_1 经过 5 个路由器连接到另一个主机 H_2 , 中间经过了 6 个网络或经过 6 跳。或者说, H_1 到 H_2 的距离是 6。在图中并没有画出网络。在讨论路由选择问题时, 在主机和路由器之间或在路由器之间的网络, 往往都用一条线段来表示。需要注意的是, 到直接连接的网络的距离也可定义为 0。但这两种不同的定义对实现协议 RIP 并无影响, 因为这对选择最佳路由的过程其实是一样的。

RIP 不能在两个网络之间同时使用多条路由。RIP 选择一条具有最少网络数的路由 (即最短路由), 哪怕还存在另一条高速 (低时延) 但网络数较多的路由。

本节讨论的 RIP 协议和下一节要讨论的 OSPF 协议, 都是分布式路由选择协议。它们的共同特点就是每一个路由器都要不断地和其他一些路由器交换路由信息。我们一定要弄清以下三个要点, 即**和哪些路由器交换信息? 交换什么信息? 在什么时候交换信息?**

协议 RIP 的特点是:

(1) **仅和相邻路由器交换信息。**如果两个路由器之间的通信不需要经过另一个路由器, 那么这两个路由器就是相邻的。协议 RIP 规定, 不相邻的路由器不交换信息。

(2) **路由器交换的信息是当前本路由器所知道的全部信息, 即自己现在的路由表。**也就是说, 交换的信息是: “我到本自治系统中所有网络的 (最短) 距离, 以及到每个网络应经过的下一跳路由器”。

(3) **按固定的时间间隔交换路由信息,**例如, 每隔 30 秒。然后路由器根据收到的路由信息更新路由表。当网络拓扑发生变化时, 路由器也及时向相邻路由器通告拓扑变化后的路由信息。网络中的主机虽然也运行协议 RIP, 但只被动地接收路由器发来的路由信息。

这里要强调一点: 路由器在**刚刚开始工作时**, 它的路由表是空的。然后路由器就得出到直接相连的几个网络的距离 (这些距离定义为 1)。接着, 每一个路由器也只和**数目非常有限**的相邻路由器交换并更新路由信息。但经过若干次的更新后, 所有的路由器最终都会知道到达本自治系统中任何一个网络的最短距离和下一跳路由器的地址。

看起来协议 RIP 有些奇怪, 因为“我的路由表中的信息要依赖于你的, 而你的信息又依赖于我的。”然而事实证明, 通过这样的方式——“我告诉别人一些信息, 而别人又告诉我一些信息。我再把我知道的更新后的信息告诉别人, 别人也这样把更新后的信息再告诉我”, 最后在自治系统中所有的节点都得到了正确的路由选择信息。在一般情况下, 协议 RIP 可以**收敛**, 并且过程也较快。“收敛”就是在自治系统中所有的节点都得到正确的路由选择信息的过程。

路由表中最主要的信息就是: 到某个网络的距离 (即最短距离), 以及应经过的下一跳地址。路由表更新的原则是找出到每个目的网络的**最短距离**。这种更新算法又称为**距离向量算法**。下面就是协议 RIP 使用的距离向量算法。

2. 距离向量算法

对每一个相邻路由器发送过来的 RIP 报文, 执行以下步骤:

(1) 对地址为 X 的相邻路由器发来的 RIP 报文, 先修改此报文中的所有项目: 把“下一跳”字段中的地址都改为 X, 并把所有的“距离”字段的值加 1 (见后面的解释 1)。每一个项目都有三个关键数据, 即: 到目的网络 Net, 距离是 d, 下一跳路由器是 X。

(2) 对修改后的 RIP 报文中的每一个项目, 进行以下步骤:

若原来的路由表中没有目的网络 Net, 则把该项目添加到路由表中 (见解释 2)。

否则 (即在路由表中有目的网络 Net, 这时就再查看下一跳路由器地址)。

若下一跳路由器地址是 X，则把收到的项目替换原路由表中的项目（见解释 3）。

否则（即这个项目是：到目的网络 Net，但下一跳路由器不是 X）。

若收到的项目中的距离 d 小于路由表中的距离，则进行更新（见解释 4），

否则什么也不做（见解释 5）。

(3) 若 3 分钟还没有收到相邻路由器的更新路由表，则把此相邻路由器记为不可达的路由器，即把距离置为 16（距离为 16 表示不可达）。

(4) 返回。

上面给出的距离向量算法的基础就是 Bellman-Ford 算法（或 Ford-Fulkerson 算法）。这种算法的要点是这样的：

设 X 是节点 A 到 B 的最短路径上的一个节点。若把路径 A→B 拆成两段路径 A→X 和 X→B，则每一段路径 A→X 和 X→B 也都分别是节点 A 到 X 和节点 X 到 B 的最短路径。

下面是对上述距离向量算法的五点解释。

解释 1：这样做是为了便于进行本路由表的更新。假设从位于地址 X 的相邻路由器发来的 RIP 报文的某一个项目是：“Net2, 3, Y”，意思是“我经过路由器 Y 到网络 Net2 的距离是 3”，那么本路由器就可推断出：“我经过 X 到网络 Net2 的距离应为 $3 + 1 = 4$ ”。于是，本路由器就把收到的 RIP 报文的这一个项目修改为“Net2, 4, X”，作为下一步和路由表中原有项目进行比较时使用（只有比较后才能知道是否需要更新）。读者可注意到，收到的项目中的 Y 对本路由器是没有用的，因为 Y 不是本路由器的下一跳路由器地址。

解释 2：表明这是新的目的网络，应当加入到路由表中。例如，本路由表中没有到目的网络 Net2 的路由，那么在路由表中就要加入新的项目“Net2, 4, X”。

解释 3：为什么要替换呢？因为这是最新的消息，要以最新的消息为准。到目的网络的距离有可能增大或减小，但也可能没有改变。例如，不管原来路由表中的项目是“Net2, 3, X”还是“Net2, 5, X”，都要更新为现在的“Net2, 4, X”。

解释 4：例如，若路由表中已有项目“Net2, 5, P”，就要更新为“Net2, 4, X”。因为到网络 Net2 的距离原来是 5，现在减到 4，更短了。

解释 5：若距离更大了，显然不应更新。若距离不变，更新后得不到好处，因此也不更新。

【例 4-4】已知路由器 R₆ 有表 4-8(a)所示的路由表。现在收到相邻路由器 R₄ 发来的路由更新信息，如表 4-8(b)所示。试更新路由器 R₆ 的路由表。

表 4-8(a) 路由器 R₆ 的路由表

目的网络	距离	下一跳路由器
Net2	3	R ₄
Net3	4	R ₅
...	...	...

表 4-8(b) R₄ 发来的路由更新信息

目的网络	距离	下一跳路由器
Net1	3	R ₁
Net2	4	R ₂
Net3	1	直接交付

【解】 如同路由器一样，我们不需要知道该网络的拓扑。

先把表 4-8(b)中的距离都加 1，并把下一跳路由器都改为 R₄，得出表 4-8(c)。

表 4-8(c) 修改后的表 4-8(b)

目的网络	距离	下一跳路由器
Net1	4	R ₄
Net2	5	R ₄
Net3	2	R ₄

把这个表的每一行和表 4-8(a)进行比较。

第一行在表 4-8(a)中没有，因此要把这一行添加到表 4-8(a)中。

第二行的 Net2 在表 4-8(a)中有，且下一跳路由器也是 R₄。因此要更新（距离增大了）。

第三行的 Net3 在表 4-8(a)中有，但下一跳路由器不同。于是就要比较距离。新的路由信息的距离是 2，小于原来表中的 4，因此要更新。

这样，得出更新后的 R₆ 的路由表如表 4-8(d)所示。

表 4-8(d) 路由器 R₆ 更新后的路由表

目的网络	距离	下一跳路由器
Net1	4	R ₄
Net2	5	R ₄
Net3	2	R ₄
...	...	...

协议 RIP 让一个自治系统中的所有路由器都和自己的相邻路由器定期交换路由信息，并不断更新其路由表，使得从每一个路由器到每一个目的网络的路由都是最短的（即跳数最少）。这里还应注意：虽然所有的路由器最终都拥有了整个自治系统的全局路由信息，但由于每一个路由器的位置不同，它们的路由表当然也应当是不同的。

现在较新的 RIP 版本是 1998 年 11 月公布的 RIP2 [RFC 2453, STD57]，新版本协议本身并无多大变化，但性能上有些改进。RIP2 可以支持无分类域间路由选择 CIDR。此外，RIP2 还提供简单的鉴别过程支持多播。

图 4-40 表明 RIP 报文作为运输层用户数据报 UDP 的数据部分进行传送（使用 UDP 的端口 520。端口的意义见 5.2.2 节）。

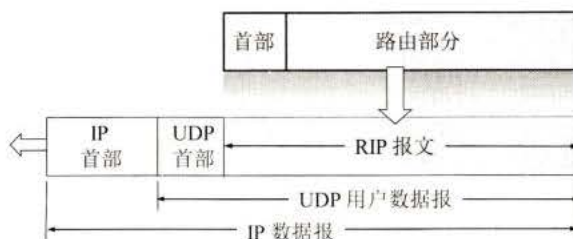


图 4-40 RIP2 的报文用 UDP 用户数据报传送

RIP 报文由首部和路由部分组成。在路由部分要填入自治系统号 ASN (Autonomous

System Number)^①，这是考虑使 RIP 有可能收到本自治系统以外的路由选择信息。还要指出目的网络地址（包括网络的子网掩码）、下一跳路由器地址以及到此网络的距离。一个 RIP 报文最多可包括 25 个路由。如超过，必须再用一个 RIP 报文来传送。

3. 坏消息传播得慢

RIP 存在的一个问题是当网络出现故障时，要经过比较长的时间才能将此信息传送到所有的路由器。我们可以用图 4-41 的简单例子来说明。设三个网络通过两个路由器互连起来，并且都已建立了各自的路由表。图中路由器交换的信息只给出了我们感兴趣的一行内容。路由器 R_1 中的“Net1, 1, 直接”表示“到网 Net1 的距离是 1，直接交付”。路由器 R_2 中的“Net1, 2, R_1 ”表示“到网 Net1 的距离是 2，下一跳经过 R_1 ”。

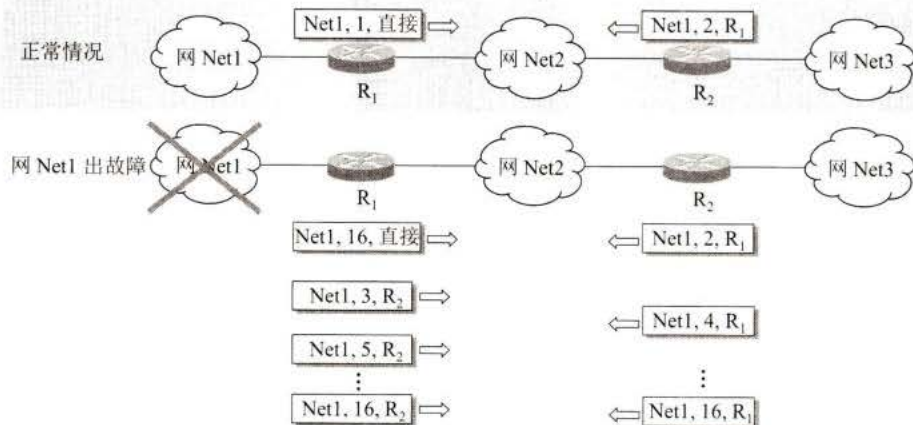


图 4-41 协议 RIP 的缺点：坏消息传播得慢

现在假定路由器 R_1 到网 Net1 的链路出了故障， R_1 无法到达网 Net1。于是路由器 R_1 把到网 Net1 的距离改为 16，表示不可达，因而在 R_1 的路由表中的相应项目变为“Net1, 16, 直接”。但是，很可能要经过 30 秒钟后 R_1 才把更新信息发送给 R_2 。然而 R_2 可能已经先把自己的路由表发送给了 R_1 ，其中有“Net1, 2, R_1 ”这一项。

R_1 收到 R_2 的更新报文后，误认为可经过 R_2 到达网 Net1，于是把收到的路由信息“Net1, 2, R_1 ”修改为：“Net1, 3, R_2 ”，表明“我到网 Net1 的距离是 3，下一跳经过 R_2 ”，并把更新后的信息发送给 R_2 。

同理， R_2 接着又更新自己的路由表为“Net1, 4, R_1 ”，以为“我到网 Net1 距离是 4，下一跳经过 R_1 ”。

这样的更新一直继续下去，直到 R_1 和 R_2 到网 Net1 的距离都增大到 16 时， R_1 和 R_2 才知道原来网 Net1 是不可达的。协议 RIP 的这一特点叫作：好消息传播得快，而坏消息传播得慢。网络出故障的传播时间往往较长（例如数分钟）。这是 RIP 的一个主要缺点。

但如果一个路由器发现了更短的路由，那么这种更新信息就传播得很快。

^① 注：自治系统号 ASN 原来规定为一个 16 位的号码（最大的号码是 65535），由 IANA 分配，具体的号码指派范围相当复杂，可从网上查出[W-ASN]。现在已经把 ASN 扩展到 32 位[RFC 6793]。自 2007 年 1 月起，32 位自治系统号 ASN 在全球互联网开始分配和使用。为了便于书写，在 32 位 ASN 中使用两段点分十进制记法，把 65535 在第 1 段中记为 1，例如，ASN 131074 可以记为 ASN 2.4。

为了使坏消息传播得更快些,可以采取多种措施。例如,让路由器记录收到某特定路由信息的接口,而不让同一路由信息再通过此接口向反方向传送。

总之,协议 RIP 最大的优点就是**实现简单,开销较小**。但协议 RIP 的缺点也较多。首先,RIP 限制了网络的规模,它能使用的最大距离为 15 (16 表示不可达)。其次,路由器之间交换的路由信息是路由器中的完整路由表,因而随着网络规模的扩大,开销也就增加。最后,“坏消息传播得慢”,使更新过程的收敛时间过长。因此,对于规模较大的网络就应当使用下一节所述的 OSPF 协议。然而目前在规模较小的网络中,使用协议 RIP 的仍占多数。

4.6.3 内部网关协议 OSPF

1. 协议 OSPF 的基本特点

这个协议的名字是**开放最短路径优先 OSPF (Open Shortest Path First)**。它是为克服 RIP 的缺点在 1989 年开发出来的。OSPF 的原理很简单,但实现起来却较复杂。“**开放**”表明 OSPF 协议不是受某一家厂商控制,而是公开发表的。“**最短路径优先**”是因为使用了 **Dijkstra 提出的最短路径算法 SPF**。现在使用的协议 OSPF 是第二个版本 OSPFv2 [RFC 2328, STD54]。关于 OSPF 可参阅专著[MOY98], [HUIT95]。

请注意:OSPF 只是一个协议的名字,它并不表示其他的路由选择协议不是“最短路径优先”。实际上,所有的在自治系统内部使用的路由选择协议(包括协议 RIP)都是要寻找一条最短的路径。

OSPF 最主要的特征就是使用**链路状态协议(link state protocol)**,而不是像 RIP 那样的距离向量协议。协议 OSPF 的特点是:

(1) 向本自治系统中**所有路由器**发送信息。这里使用的方法是**洪泛法(flooding)**,这就是路由器通过所有输出端口向所有相邻的路由器发送信息。而每一个相邻路由器又再将此信息发往其所有的相邻路由器(但不再发送给刚刚发来信息的那个路由器)。这样,最终整个区域中所有的路由器都得到了这个信息的一个副本。更具体的做法后面还要讨论。我们应注意,协议 RIP 是仅仅向自己相邻的几个路由器发送信息。

(2) 发送的信息就是与本路由器**相邻的所有路由器的链路状态**,但这只是路由器所知道的部分信息。所谓“**链路状态**”就是说明本路由器都和哪些路由器相邻^①,以及该链路的“**度量**”(metric)。OSPF 将这个“度量”用来表示费用、距离、时延、带宽,等等。这些都由网络管理人员来决定,因此较为灵活。有时为了方便就称这个度量为“**代价**”。我们应注意,对于协议 RIP,发送的信息是:“到所有网络的距离和下一跳路由器”。

(3) 当链路状态发生变化或每隔一段时间(如 30 分钟),路由器向所有路由器用洪泛法发送链路状态信息。

从上述的前两点可以看出,OSPF 和 RIP 的工作原理相差较大。

由于各路由器之间频繁地交换链路状态信息,因此所有的路由器最终都能建立一个**链路状态数据库(link-state database)**,这个数据库实际上就是全网的拓扑结构图。这个拓扑结构图在全网范围内是一致的(这称为**链路状态数据库的同步**)。因此,每一个路由器都知道

扫一扫



视频讲解

^① 注:在前面我们已经说过,在讨论路由器之间是如何交换路由信息时,最好将路由器之间的网络简化为一条链路。

OSPF 的“链路状态”中的“链路”实际上就是指“和这两个路由器都有接口的网络”。

全网共有多少个路由器，以及哪些路由器是相连的，其代价是多少，等等。每一个路由器使用链路状态数据库中的数据，构造出自己的路由表（例如，使用 Dijkstra 的最短路径路由算法）。我们注意到，协议 RIP 的每一个路由器虽然知道到所有的网络的距离以及下一跳路由器，但却不知道全网的拓扑结构（只有到了下一跳路由器，才能知道再下一跳应当怎样走）。

OSPF 的链路状态数据库能较快地进行更新，使各个路由器能及时更新其路由表。OSPF 的更新过程收敛得快是其重要优点。

为了使 OSPF 能够用于规模很大的网络，OSPF 将一个自治系统再划分为若干个更小的范围，叫作区域(area)。图 4-42 就表示一个自治系统划分为四个区域。每一个区域都有一个 32 位的区域标识符（用点分十进制形式表示）。当然，一个区域也不能太大，在一个区域内的路由器最好不超过 200 个。

划分区域的好处就是把利用洪泛法交换链路状态信息的范围局限于每一个区域而不是整个的自治系统，这就减少了整个网络上的通信量。在一个区域内部的路由器只知道本区域的完整网络拓扑，而不知道其他区域的网络拓扑的情况。为了使每一个区域能够和本区域以外的区域进行通信，OSPF 使用层次结构的区域划分。在上层的区域叫作主干区域(backbone area)。主干区域的标识符规定为 0.0.0.0。主干区域的作用是用来连通其他在下层的区域。从其他区域来的信息都由区域边界路由器(area border router)进行概括。在图 4-42 中，路由器 R₃、R₄ 和 R₇ 都是区域边界路由器，而显然，每一个区域至少应当有一个区域边界路由器。在主干区域内的路由器叫作主干路由器(backbone router)，如 R₃、R₄、R₅、R₆ 和 R₇。一个主干路由器可以同时是区域边界路由器，如 R₃、R₄ 和 R₇。在主干区域内还要有一个路由器专门和本自治系统外的其他自治系统交换路由信息。这样的路由器叫作自治系统边界路由器（如图 4-42 中的 R₆）。

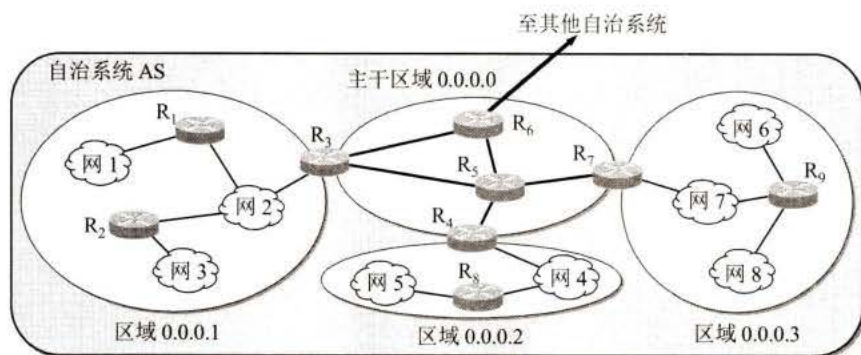


图 4-42 OSPF 划分为两种不同的区域

采用分层次划分区域的方法虽然使交换信息的种类增多了，同时也使 OSPF 协议更加复杂了。但这样做却能使每一个区域内部交换路由信息的通信量大大减小，因而使 OSPF 协议能够用于规模很大的自治系统中。这里，我们再一次地看到划分层次在网络设计中的重要性。

除了以上的几个基本特点外，OSPF 还具有下列的一些特点：

(1) OSPF 允许管理员给每条路由指派不同的代价。例如，高带宽的卫星链路对于非实时的业务可设置为较低的代价，但对于时延敏感的业务就可设置为非常高的代价。因此，OSPF 对于不同类型的业务可计算出不同的路由。链路的代价可以是 1 至 65535 中的任何一个无量纲的数，因此十分灵活。商用的网络在使用 OSPF 时，通常根据链路带宽来计算链路

的代价。这种灵活性是 RIP 所没有的。

(2) 如果到同一个目的网络有多条相同代价的路径，那么可以将通信量分配给这几条路径。这叫作多路径间的**负载均衡**(load balancing)。在代价相同的多条路径上分配通信量是通信量工程中的简单形式。RIP 只能找出到某个网络的一条路径。

(3) 所有在 OSPF 路由器之间交换的分组（例如，链路状态更新分组）都具有**鉴别**的功能，因而保证了仅在可信赖的路由器之间交换链路状态信息。

(4) OSPF 支持可变长度的子网划分和无分类的编址 CIDR。

(5) 由于网络中的链路状态可能经常发生变化，因此 OSPF 让每一个链路状态都带上一个 32 位的**序号**，序号越大状态就越新。OSPF 规定，链路状态序号增长的速率不得超过每 5 秒钟 1 次。这样，全部序号空间在 600 年内不会产生重复号。

2. OSPF 的五种分组类型

OSPF 共有以下五种分组类型：

(1) **类型 1，问候(Hello)分组**，用来发现和维持邻站的可达性。

(2) **类型 2，数据库描述(Database Description)分组**，向邻站给出自己的链路状态数据库中的所有链路状态项目的摘要信息。

(3) **类型 3，链路状态请求(Link State Request)分组**，向对方请求发送某些链路状态项目的详细信息。

(4) **类型 4，链路状态更新(Link State Update)分组**，用洪泛法对全网更新链路状态。这种分组是最复杂的，也是 OSPF 协议最核心的部分。路由器使用这种分组将其链路状态通知给邻站。链路状态更新分组共有五种不同的链路状态[RFC 2328]，这里从略。

(5) **类型 5，链路状态确认(Link State Acknowledgment)分组**，对链路更新分组的确认。

OSPF 分组是作为 IP 数据报的数据部分来传送的（如图 4-43 所示）。OSPF 不用 UDP 而是**直接用 IP 数据报传送**（其 IP 数据报首部的协议字段值为 89）。OSPF 构成的数据报很短。这样做可减少路由信息的通信量。数据报很短的另一好处是可以不必将长的数据报分片传送。分片传送的数据报只要丢失一个，就无法组装成原来的数据报，而整个数据报就必须重传。



图 4-43 OSPF 分组用 IP 数据报传送

OSPF 规定，每两个相邻路由器每隔 10 秒钟要交换一次问候分组。这样就能确知哪些邻站是可达的。对相邻路由器来说，“可达”是最基本的要求，因为只有可达邻站的链路状态信息才存入链路状态数据库（路由表就是根据链路状态数据库计算出来的）。在正常情况下，网络中传送的绝大多数 OSPF 分组都是问候分组。若有 40 秒钟没有收到某个相邻路由器发来的问候分组，则可认为该相邻路由器是不可达的，应立即修改链路状态数据库，并重新计算路由表。

其他的四种分组都是用来进行链路状态数据库的同步。所谓**同步**就是指不同路由器的

链路状态数据库的内容是一样的。两个同步的路由器叫作“完全邻接的”(fully adjacent)路由器。不是完全邻接的路由器表明它们虽然在物理上是相邻的,但其链路状态数据库并没有达到一致。

当一个路由器刚开始工作时,它只能通过问候分组得知它有哪些相邻的路由器在工作,以及将数据发往相邻路由器所需的“代价”。如果所有的路由器都把自己的本地链路状态信息对全网进行广播,那么各路由器只要将这些链路状态信息综合起来就可得出链路状态数据库。但这样做开销太大,因此 OSPF 采用下面的办法。

OSPF 让每一个路由器用数据库描述分组和相邻路由器交换本数据库中已有的链路状态摘要信息。摘要信息主要就是指出有哪些路由器的链路状态信息(以及其序号)已经写入了数据库。经过与相邻路由器交换数据库描述分组后,路由器就使用链路状态请求分组,向对方请求发送自己所缺少的某些链路状态项目的详细信息。通过一系列的这种分组交换,全网同步的链路数据库就建立了。

在网络运行的过程中,只要一个路由器的链路状态发生变化,该路由器就要使用链路状态更新分组,用洪泛法向全网更新链路状态。OSPF 使用的是可靠的洪泛法,其要点如图 4-44 所示。设路由器 R 用洪泛法发出链路状态更新分组。图中用一些小的箭头表示更新分组。第一次先发给相邻的三个路由器。这三个路由器将收到的分组再进行转发时,要将其上游路由器除外。可靠的洪泛法是在收到更新分组后要发送确认(收到重复的更新分组只需要发送一次确认)。图中的空心箭头表示确认分组。

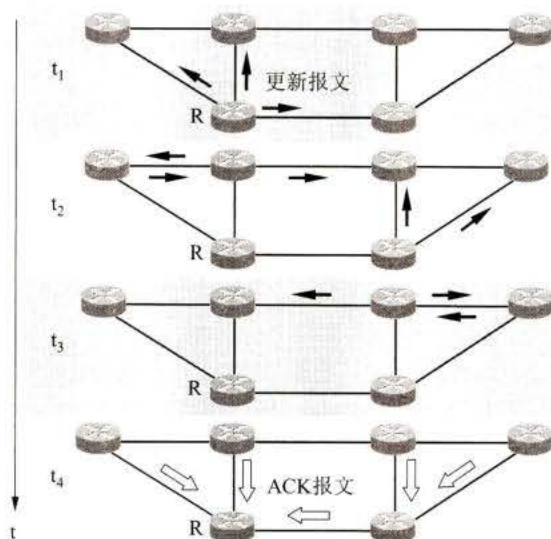


图 4-44 用可靠的洪泛法发送更新分组

为了确保链路状态数据库与全网的状态保持一致,OSPF 还规定每隔一段时间,如 30 分钟,要刷新一次数据库中的链路状态。

由于一个路由器的链路状态只涉及与相邻路由器的连通状态,因而与整个互联网的规模并无直接关系。因此当互联网规模很大时,OSPF 协议要比距离向量协议 RIP 好得多。由于 OSPF 没有“坏消息传播得慢”的问题,据统计,其响应网络变化的时间小于 100 ms。

若 N 个路由器连接在一个以太网上,则每个路由器要向其他 $(N - 1)$ 个路由器发送链路状态信息,因而共有 $N(N - 1)$ 个链路状态要在这个以太网上传送。OSPF 协议对这种多点接

入的局域网采用了指定的路由器(designated router)的方法,使广播的信息量大大减少。指定的路由器代表该局域网上所有的链路向连接到该网络上的各路由器发送状态信息。

4.6.4 外部网关协议 BGP

1. 协议 BGP 的主要特点

在外部网关协议(或边界网关协议)BGP 中,现在使用的是第 4 个版本 BGP-4(常简称为 BGP)。虽然最近陆续发布了不少 BGP-4 的更新文档,但目前 BGP-4 仍然是草案标准[RFC 4271]。协议 BGP 对互联网非常重要。我们知道,前面两节所介绍的路由选择协议 RIP 和 OSPF,都只能在一个自治系统 AS 内部工作。因此,若没有协议 BGP,那么分布在全世界数以万计的 AS 都将是一个个没有联系的孤岛。正是由于有了 BGP 这种黏合剂,才使得这么多的 AS 孤岛能够连接成一个完整的互联网。从这个意义上考虑,协议 BGP 应当是所有路由选择协议中最为重要的一个。

我们首先应当弄清,在不同自治系统 AS 之间的路由选择为什么不能使用前面讨论过的内部网关协议,如 RIP 或 OSPF?

我们知道,内部网关协议(如 RIP 或 OSPF)主要是设法使数据报在一个 AS 中尽可能有效地从源站传送到目的站。在一个 AS 内部也不需要考虑其他方面的策略。然而 BGP 使用的环境却不同。这主要是因为以下的两个原因:

第一,互联网的规模太大,使得自治系统 AS 之间路由选择非常困难。连接在互联网主干网上的路由器,必须对任何有效的 IP 地址都能在转发表中找到匹配的网络前缀。目前在互联网的主干网路由器中,一个转发表的项目数甚至可达到 50 万个网络前缀。如果使用链路状态协议,则每一个路由器必须维持一个很大的链路状态数据库。对于这样大的主干网用 Dijkstra 算法计算最短路径时花费的时间也太长。另外,由于自治系统 AS 各自运行自己选定的内部路由选择协议,并使用本 AS 指定的路径度量,因此,当一条路径通过几个不同 AS 时,要想对这样的路径计算出有意义的代价是不太可能的。例如,对某 AS 来说,代价为 1000 可能表示一条比较长的路由。但对另一 AS,代价为 1000 却可能表示不可接受的坏路由。因此,对于自治系统 AS 之间的路由选择,要用“代价”作为度量来寻找最佳路由也是很很不现实的。比较合理的做法是在自治系统之间交换“可达性”信息(即“可到达”或“不可到达”)。例如,告诉相邻路由器:“到达网络前缀 N 可经过自治系统 AS_x ”。

第二,自治系统 AS 之间的路由选择必须考虑有关策略。由于相互连接的网络的性能相差很大,根据最短距离(即最少跳数)找出来的路径,可能并不合适。也有的路径的使用代价很高或很不安全。还有一种情况,如自治系统 AS_1 要发送数据报给自治系统 AS_2 ,本来最好是经过自治系统 AS_3 。但 AS_3 不愿意让这些数据报通过本自治系统的网络,即使 AS_1 愿意付一定的费用。但另一方面,自治系统 AS_3 愿意让某些相邻自治系统的数据报通过自己的网络,特别是对那些付了服务费的某些自治系统更是如此。因此,自治系统之间的路由选择协议应当允许使用多种路由选择策略。这些策略包括政治、安全或经济方面的考虑。例如,我国国内的站点在互相传送数据报时不应经过国外圈子,特别是,不要经过某些对我国的安全有威胁的国家。这些策略都是由网络管理人员对每一个路由器进行设置的,但这些策略并不是自治系统之间的路由选择协议本身。还可举出一些策略的例子,如:“仅在到达下列这些地址时才经过 AS_x ”,“ AS_x 和 AS_y 相比时应优先通过 AS_x ”,等等。显然,使用这些策



略是为了找出较好的路由而不是最佳路由。

由于上述情况，边界网关协议 BGP 只能是力求选择出一条能够到达目的网络前缀且比较好的路由（不能兜圈子），而非非要计算出一条最佳路由。这里所说的 BGP 路由，是指经过哪些自治系统 AS 可以到达目的网络前缀。当然，这选择出的比较好的路由，也有时不严格地称为最佳路由。BGP 采用了路径向量(path vector)路由选择协议，它与距离向量协议（如 RIP）和链路状态协议（如 OSPF）都有很大的区别。

2. BGP 路由

在一个自治系统 AS 中有两种不同功能的路由器，即边界路由器（或边界网关）和内部路由器。一个 AS 至少要有有一个边界路由器和相邻 AS 的边界路由器直接相连。在讨论协议 BGP 时，应特别注意边界路由器的作用。正是由于有了边界路由器，AS 之间才能利用协议 BGP 交换可达性路由信息。

当两个边界路由器（例如图 4-45(a)中的 R_1 和 R_2 ）进行通信时，必须先建立 TCP 连接（端口号为 179，TCP 连接将在第 5 章中学习），这种 TCP 连接又称为半永久性连接（即双方交换完信息后仍然保持着连接状态）。像 R_1 和 R_2 之间的这种连接称为 eBGP 连接，但通常就简称为 eBGP，e 表示外部 external。现在，边界路由器 R_1 可通过 eBGP 向对等端 R_2 发送 BGP 路由“X, AS₁, R_1 ”，意思是“从 R_1 经 AS₁ 可到达 X”。这样，通过 eBGP 连接，AS₂ 中的边界路由器 R_2 就知道了到达 AS₁ 中的前缀 X 的 BGP 路由。

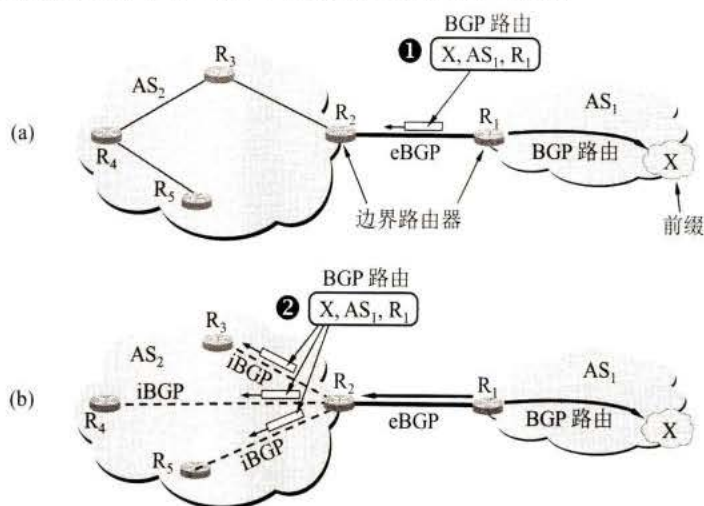


图 4-45 AS 之间的 eBGP 连接(a)和 AS 内部的 iBGP 连接(b)

但是，仅有边界路由器 R_2 知道“到 AS₁ 的前缀 X 的 BGP 路由”是远远不够的。边界路由器 R_2 应当把获得的 BGP 路由，再转发给 AS 内部的其他路由器。为此，协议 BGP 规定，在 AS 内部，两个路由器之间还需要建立 iBGP（也就是 iBGP 连接，i 表示内部 internal），iBGP 也使用 TCP 连接传送 BGP 报文。图 4-45(b)中表示边界路由器 R_2 在三个 iBGP 连接上，向 AS₂ 内部的其他三个路由器转发自己收到的 BGP 路由。至此，AS₂ 内的所有路由器都知道了这条 BGP 路由信息。由此可见，协议 BGP 并非仅运行在 AS 之间，而且也要运行在 AS 的内部。

协议 BGP 规定，在一个 AS 内部所有的 iBGP 必须是全连通的。即使两个路由器之间没

有物理连接，但它们之间仍然有 iBGP 连接（如图 4-46 所示）。



图 4-46 在 AS 内路由器之间的物理连接与 iBGP 连接

这里要说明一下。图 4-45(a)中的边界路由器 R_1 通告给边界路由器 R_2 的 BGP 路由可能有很多条（在图中只画出了一条）。但 R_2 根据本 AS 管理员所规定的策略，可以拒绝某些路由（收到这种路由后即删除掉），而在 iBGP 连接上仅转发符合规定策略的 BGP 路由。

我们还需要指出，虽然 eBGP 和 iBGP 的最后一个字母 P 是代表“协议(Protocol)”，但 eBGP 和 iBGP 并不是两个不同的协议。根据 RFC 4271，eBGP 是在不同 AS 的两个对等端之间的 BGP 连接，而 iBGP 是同一 AS 的两个对等端之间的 BGP 连接。在这两种不同连接上传送的 BGP 报文，都遵循同样的协议 BGP，使用同样的报文格式和具有同样的属性类型。唯一的不同点就是在发送 BGP 路由通告时的规则有所不同。这就是，从 eBGP 对等端收到的 BGP 路由，可通过 iBGP 告诉同一 AS 内的对等端。反过来也是可以的，即从 iBGP 对等端收到的 BGP 路由，可通过 eBGP 告诉在不同 AS 的对等端。但是，从 iBGP 对等端收到 BGP 路由，不能转告给同一个 AS 内不同 iBGP 的对等端。

图 4-47 形象地说明了 eBGP 和 iBGP 的作用。图中画出了四个自治系统 AS，每一个 AS 都必须运行本 AS 选择的内部网关协议 IGP，例如 OSPF 或 RIP。而协议 BGP 是在 iBGP 连接和 eBGP 连接之上运行的。

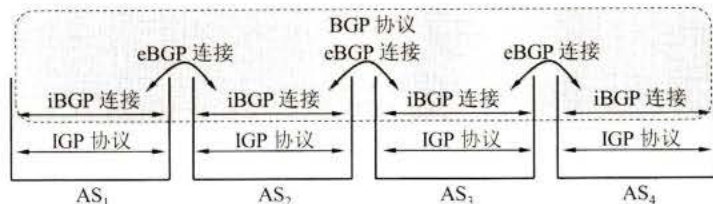


图 4-47 协议 BGP 在 iBGP 和 eBGP 之上运行

扫一扫



视频讲解

后面我们经常要讲到 BGP 路由，因此下面介绍一下 BGP 路由的一般格式。前面例子中的 BGP 路由正是按照这种格式书写的。

BGP 路由 = “前缀, BGP 属性” = “前缀, AS-PATH, NEXT-HOP”

前缀的意思很明确，就是通告的 BGP 路由终点（子网前缀）。

BGP 属性有好几种类型，但最重要两个就是这里列出的 AS-PATH 和 NEXT-HOP。

AS-PATH（自治系统路径）是通告的 BGP 路由所经过的自治系统。BGP 路由每经过一个 AS，就将其自治系统号 ASN 加入到 AS-PATH 中（本节在举例中没有使用具体的 ASN 数字号码，而是用 AS_1 , AS_2 等符号来表示不同的 AS）。从这里可以清楚地看出，“BGP 路由”必须指出通过哪些自治系统 AS，但不指出路由中途要通过哪些路由器。

NEXT-HOP（下一跳）是通告的 BGP 路由起点。

细心的读者会发现，路由问题并未都解决完。例如，在图 4-48 中 AS_2 内的路由器 R_4 在收到 BGP 路由“X, AS_1 , R_1 ”后，知道了“从 R_1 出发就能到达 AS_1 中的前缀 X”。但路由器

R₄ 应当怎样构造自己的转发表呢？这需要经过两次递归查找。

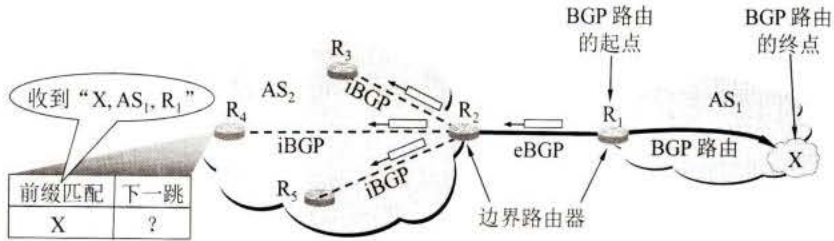


图 4-48 BGP 路由的起点、经过的 AS 和终点

首先，R₄ 要把这条 BGP 路由的起点进行转换。原来的 BGP 路由是“R₁→X”，路由的起点 R₁ 并不在 AS₂ 中。AS₂ 中的路由器都不能识别 R₁。因此现在 R₄ 要把 BGP 路由的起点改为 R₁ 的对等端 R₂，把 BGP 路由变为“R₂→R₁→X”。由于 R₂ 位于 AS₂ 中，因此 AS₂ 里面的所有路由器都能把分组转发到 R₂，然后就能再经过 R₁，最后到达前缀 X。

其次，R₄ 要利用内部网关协议，找到从 R₄ 到 R₂ 的最佳路由中的下一跳。在本例中，查出下一跳是 R₃。于是 R₄ 在转发表中增加了到达前缀 X 的下一跳是 R₃ 这一项目。

前缀匹配	下一跳	注释
X	R ₃	转发表中有下一跳 R ₃ 表示前缀 X 可达。

用类似的方法，路由器 R₃ 也在自己的转发表中增加了到达前缀 X 的项目。

前缀匹配	下一跳	注释
X	R ₂	转发表中有下一跳 R ₂ 表示前缀 X 可达。

这样，路由器 R₄ 只要收到要到达前缀 X 的分组，都按照 R₄→R₃→R₂→R₁→X 的路径，最后到达前缀 X。

总之，每一个路由器收到一条新的 BGP 路由通告后，必须经过上述步骤，才能在自己的转发表中，增加到达终点的“下一跳”的相应项目。

在实际的转发表中，“前缀匹配”项目都用 CIDR 记法表示。由于路由器有两个以上的接口，因此“下一跳”项目用进入该路由器的接口的 IP 地址来表示。在讲解协议 BGP 的原理时，我们就用更简洁的符号来表示。

3. 三种不同的自治系统 AS

在互联网中自治系统 AS 的数量非常之多，其连接图也是相当复杂的。但归纳起来，可以把 AS 划分为图 4-49 中所示的三大类，即末梢 AS (stub AS)、穿越 AS (transit AS) 和对等 AS (peering AS)。

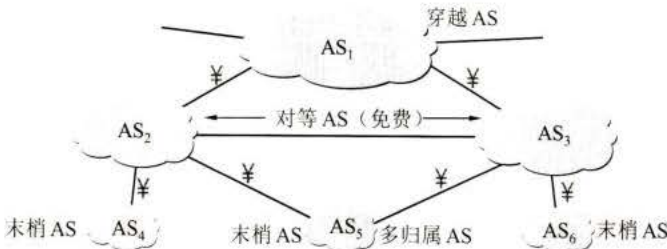


图 4-49 几种不同类型的 AS

末梢 AS 是比较小的 AS (如图中的 AS₄, AS₅ 和 AS₆)，其特点是这些 AS 或者把分组发送给其直接连接的 AS，或者从其直接连接的 AS 接收分组，但不会把来自其他 AS 的分组再转发到另一个 AS。末梢 AS 必须向所连接的 AS 付费才能发送或接收分组。图中链路旁边的人民币符号¥表示需要对转发分组付费。

末梢 AS 也可以同时连接到两个或两个以上的 AS (如图 4-49 中的 AS₅)。这种末梢 AS 就称为多归属 AS (multihomed AS)。多归属 AS 可以增加连接的可靠性，因为若有一条连接出现故障，那么还有另一条连接可用。作为末梢 AS 的 AS₅ 不能把 AS₃ 发送过来的分组转发到 AS₂。同理，AS₅ 也不能把 AS₂ 发送过来的分组转发到 AS₃。这就是说，末梢 AS 不是穿越 AS，它不允许分组穿越自己的自治系统。末梢 AS₅ 也不能把 (AS₅→AS₂→AS₄) 这样的 BGP 路由信息通告给 AS₃。如果 AS₃ 有分组要转发给 AS₄，可以通过对等 AS₂ 转发，但不能通过末梢 AS₅。

如图 4-49 所示的穿越 AS₁ 往往是拥有很好的高速通信干线的主干 AS，其任务就是为其他的 AS 有偿转发分组。通常都会有很多的 AS 连接到穿越 AS 上。

对等 AS (如图 4-49 中的 AS₂ 和 AS₃) 是经过事先协商的两个 AS，彼此之间的发送或接收分组都不收费，这样大家转发分组都比较方便。

这里我们要强调一下，BGP 路由必须避免兜圈子的出现。现观看图 4-49 所示的几个自治系统 AS。AS₃ 向 AS₁ 通告可到达 AS₆ 的 BGP 路由中的属性 AS-PATH 是 [AS₃ AS₆]，AS₁ 在收到的 BGP 路由的属性 AS-PATH 中的最前面，添加上自己的 AS，通报给 AS₂：[AS₁ AS₃ AS₆]。AS₂ 收到 BGP 路由后，向 AS₃ 通报 BGP 路由属性 AS-PATH 是 [AS₂ AS₁ AS₃ AS₆]。当 AS₃ 收到 BGP 路由后，检查属性 AS-PATH 序列中已经有了自己的 AS₃，如果 AS₃ 接受这个路由，并添加上本 AS 号，则将在属性 AS-PATH 中出现两个 AS₃ (见有灰色底纹的两个 AS₃)：[AS₃ AS₂ AS₁ AS₃ AS₆]。这就构成了一个兜圈子的 BGP 路由，因此 AS₃ 应立即删除此 BGP 路由，因而避免了兜圈子路由的出现。请记住，在属性 AS-PATH 中不允许出现相同的 AS。

AS₃ 还可向 AS₂ 通报可到达 AS₆ 的另一条 BGP 路由，其 AS-PATH 是 [AS₃ AS₆]。因此 AS₂ 知道有两条 BGP 路由可到达 AS₆，其 AS-PATH 是 [AS₂ AS₁ AS₃ AS₆] 和 [AS₂ AS₃ AS₆]。

4. BGP 的路由选择

假如从一个 AS 到另外一个 AS 中的前缀 X 只有一条 BGP 路由，那么就不存在选择 BGP 路由的问题，因为这时 BGP 路由是唯一的。

但如果到前缀 X 有两条或更多的 BGP 路由可供选择，那么就应当根据以下的原则，按照这里给出的先后顺序，选择一条较好的 BGP 路由。

- 本地偏好 LOCAL-PREF (LOCAL PReference) 值最高的路由要首先选择。

在 BGP 路由中的属性里面有一个选项叫作本地偏好，在属性中记为 LOCAL-PREF。本地偏好也就是本地优先，“本地”的意思是指，从本 AS 开始的、到同一个前缀的不同 BGP 路由中，挑选一个较好的 (即偏好值最高的) 路由。这可由路由器管理员或网络管理员根据政治上或经济上的策略来设置。

例如在图 4-50 中，AS₁ 分别用高速和低速链路连接到 AS₂ 和 AS₃，并从这两个 AS 获悉，可通过 AS₂ 或 AS₃ 到达 AS₄。但 AS₁ 认为，若有分组要从 AS₁ 转发到 AS₄，应优先选择路由器 R₁ 离开 AS₁。于是就把从 R₁ 离开 AS₁ 的 BGP 路由的属性 LOCAL-PREF 值设为 300，而

把从 R_2 离开 AS_1 的 BGP 路由的 LOCAL-PREF 值设为 200。这一信息通过 iBGP 通告 AS_1 内部的所有路由器。这样，凡是有分组要转发到 AS_4 ，都优先选择从 R_1 离开 AS_1 。

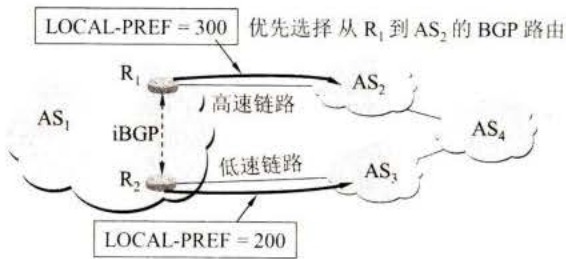


图 4-50 LOCAL-PREF 值较高的路由优先

但是，即使所有的通信量都通过这条高速链路，使得链路负荷过重，协议 BGP 也无法把一些负载调整到负载较轻的那条低速链路上。

如果从几条 BGP 路由中找不出本地偏好值最高的路由，则执行下一条。

- 选择具有 AS 跳数最少的路由。

现观察图 4-51 的例子。从 AS_1 到 AS_5 共有两条 BGP 路由，即 $AS_1 \rightarrow AS_2 \rightarrow AS_3 \rightarrow AS_5$ 和 $AS_1 \rightarrow AS_4 \rightarrow AS_5$ 。根据选择具有 AS 跳数最少的原则，我们应选择只通过 1 个 AS 的 BGP 路由，即 $AS_1 \rightarrow AS_4 \rightarrow AS_5$ 。但是没有想到，分组在 AS_4 中反而要经过更多次数的转发（或许 AS_4 是个很大的 AS），可能要花费更长的时间。可见选择经过 AS 数量最少的路由 $AS_1 \rightarrow AS_4 \rightarrow AS_5$ 未必更好。这个例子再次说明了协议 BGP 无法选择出最佳路由。

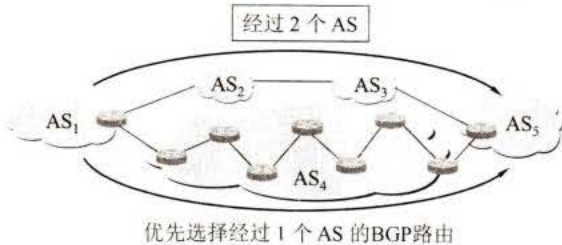


图 4-51 根据经过 AS 跳数的多少选择 BGP 路由

- 使用热土豆路由选择算法。

如果按前两种方法都无法选择最好的路由，那么就在要进入 BGP 路由的 AS，执行热土豆路由选择算法。例如在图 4-52 中，从 AS_1 出发，共有两条 BGP 路由可到达 AS_3 ：

BGP 路由 1：从 R_3 离开 AS_1 ，然后进入 AS_2 ，再到 AS_3 。

BGP 路由 2：从 R_4 离开 AS_1 ，然后进入 AS_4 ，再到 AS_3 。

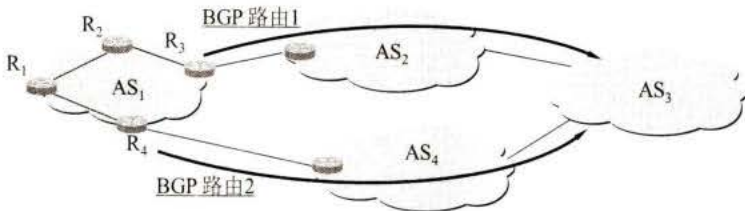


图 4-52 热土豆路由选择算法的应用举例（图中的连接都是物理连接）

假定这两条 BGP 路由的本地偏好相同，同时所经过的 AS 个数也相同。在这种情况下，

AS₁ 中的每一个路由器, 就应采用热土豆路由选择算法。这种算法把分组比喻为烫手的热土豆, 要尽快地转发出去。对于图 4-52 的例子, 就是要使分组尽快离开 AS₁, 而不考虑从哪个路由器离开 AS₁。或者说, 要让分组经过最少的转发次数离开本 AS。这时要使用内部网关协议 (如协议 OSPF 或 RIP)。对于不同的路由器, 得出的选择结果是不同的。

例如, 对于 AS₁ 中的路由器 R₁, 若要使转发的分组尽快离开 AS₁, 应选择 R₄ 作为其下一跳, 因此应选择 BGP 路由 2。这样, R₁ 的转发路径应当是: R₁→R₄→BGP 路由 2。

同理, 对于 R₂, 若要使转发的分组尽快离开 AS₁, 应选择 R₃ 作为其下一跳, 因此应选择 BGP 路由 1。这样, R₂ 的转发路径应当是: R₂→R₃→BGP 路由 1。

- 选择路由器 BGP 标识符的数值最小的路由。

当以上几种方法都无法找出最好的 BGP 路由时, 就可使用 BGP 标识符来选择路由。在 BGP 进行交互的报文中, 其首部有一 4 字节的字段, 叫作 BGP 标识符, 记为 BGP ID。这个字段被赋予一个无符号整数作为运行 BGP 的路由器的唯一标识符。具有多个接口的路由器有多个 IP 地址。BGP ID 就使用该路由器的 IP 地址中数值最大的一个。

5. BGP 的四种报文

在协议 BGP 刚运行时, BGP 连接的对等端要相互交换整个的 BGP 路由表。但以后只需要在 BGP 路由发生变化时, 才更新有变化的部分。这样做对节省网络带宽和减少路由器的处理开销方面都有好处。

在 RFC 4271 中规定了 BGP-4 的四种报文:

- (1) OPEN (打开) 报文, 用来与 BGP 连接对等端建立关系。
- (2) UPDATE (更新) 报文, 用来通告某一路由的信息, 以及列出要撤销的路由。
- (3) KEEPALIVE (保活) 报文, 用来周期性地证实与对等端的连通性。
- (4) NOTIFICATION (通知) 报文, 用来发送检测到的差错。

OPEN 报文是两个路由器之间建立了 TCP 连接后接着就必须发送的报文。OPEN 报文的作用是相互识别对方, 协商一些协议参数 (如计时器的时间)。收到 OPEN 报文的路由器, 就发回 KEEPALIVE 报文表示接受建立 BGP 连接。

UPDATE 报文是 BGP 协议的核心, 用来撤销它以前曾经通知过的路由, 或宣布增加新的路由。撤销路由可以一次撤销许多条, 但增加新路由时, 每个更新报文只能增加一条。

虽然 BGP 连接的两端建立了 TCP 连接, 传输报文是可靠的, 但 TCP 上层的 BGP 是否始终正常工作还无法确知。在对等端之间定期传送 BGP 路由表是不可取的, 因为 BGP 路由表往往过于庞大, 这样做会使网络的通信量过大。因此协议 BGP 采用的方法是让 BGP 连接的两个对等端之间, 周期性地交换 KEEPALIVE 报文, 以表示协议工作正常。KEEPALIVE 报文只包含 BGP 报文的通用首部 (19 字节长), 因此不会在网络上产生多少开销。

每个路由器都有一个保持时间计时器 (Hold Timer)。路由器每收到一个 BGP 报文, 这个计时器就重置一次, 继续从 0 开始计时。如果在商定的保持时间内没有收到对等端发来的任何一种 BGP 报文, 就认为对方已经不能工作了。发送 KEEPALIVE 报文的时间间隔取为双方事先商定的保持时间 (Hold Time) 的 1/3。例如, 在 BGP 连接建立阶段, 双方商定保持时间为 180 秒, 那么 KEEPALIVE 报文就每隔 60 秒发送一次。如果两个对等端选择的保持时间不一致, 就选择数值较小的一个作为彼此使用的保持时间。保持时间也可选择为 0。在这种情况下就永远不发送 KEEPALIVE 报文, 表明这条 BGP 连接总是正常工作的。

BGP 可以很容易地解决距离向量路由选择算法中的“坏消息传播得慢”这一问题。当某

个路由器或链路出故障时，可以从不止一个邻站获得路由信息，因此很容易选择出新的路由。

在 ASN 升级到 4 字节后，号码 ASN 的范围从 0 ~ 65535 扩大到了 0 ~ 4294967295。这样就在互联网中存在两种不同的 BGP 报文。旧 BGP 报文使用 2 字节 ASN，而新 BGP 报文使用 4 字节的 ASN，这时 BGP 路由中的路径属性记为 AS4-PATH。使用新的和旧的 BGP 报文的路由器若要进行通信，必须解决如何正确识别其 ASN 的问题[RFC 6793]。

BGP 报文是作为 TCP 报文的数据部分来传送的（如图 4-53 所示）。四种类型的 BGP 报文具有同样的首部。

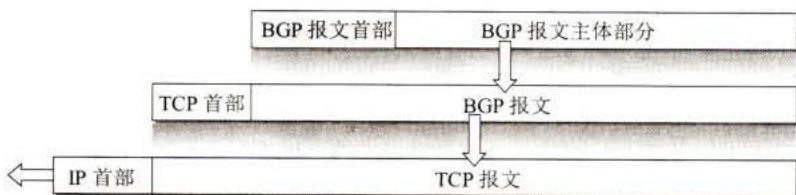


图 4-53 BGP 报文用 TCP 报文传送

在讨论完路由选择之后，我们再来介绍路由器的构成。

4.6.5 路由器的构成

1. 路由器的结构

路由器是一种具有多个输入端口和多个输出端口的专用计算机，其任务是转发分组。从路由器某个输入端口收到的分组，按照分组要去的目的地（即目的网络），把该分组从路由器的某个合适的输出端口转发给下一跳路由器。下一跳路由器也按照这种方法处理分组，直到该分组到达终点为止。路由器的转发分组正是网络层的主要工作。图 4-54 给出了一种典型的路由器的构成框图。



视频讲解

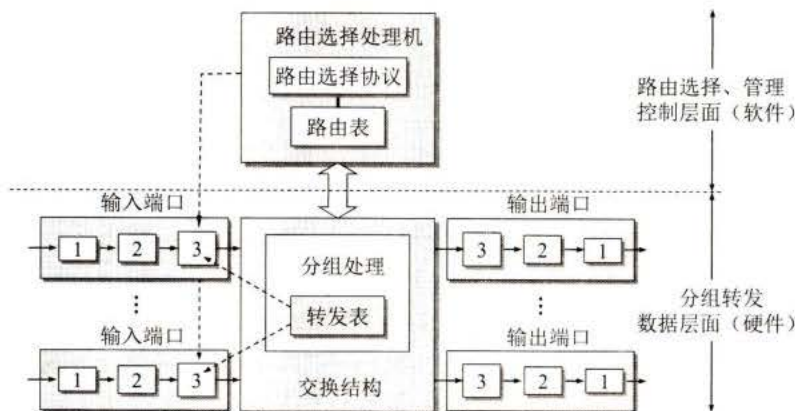


图 4-54 典型的路由器的结构（图中的数字 1~3 表示相应层次的构件）

从图 4-54 可以看出，整个的路由器结构可划分为两大部分：路由选择部分和分组转发部分。

路由选择部分也叫作控制部分，或控制层面，其核心构件是路由选择处理机。路由选择处理机的任务是根据所选定的路由选择协议构造出路由表，同时经常或定期地和相邻路由

器交换路由信息而不断地更新和维护路由表。

分组转发部分是本节所要讨论的问题，也就是**数据层面**，它由三部分组成：**交换结构**、**一组输入端口**和**一组输出端口**（请注意：这里的端口就是硬件接口）。小型路由器的端口只有几个。但某些 ISP 使用的边缘路由器的高速 10 Gbit/s 端口，则可以有多达几百个之多。下面分别讨论每一部分的组成。例如，Juniper（瞻博网络）公司的边缘路由器 MX2020 的 10 Gbit/s 端口就有 960 个。路由器整个的系统容量为 80 Tbit/s。

交换结构(switching fabric)又称为**交换组织**，它的作用就是根据**转发表(forwarding table)**对分组进行处理，将某个输入端口进入的分组从一个合适的输出端口转发出去。交换结构本身就是一种网络，但这种网络完全包含在路由器之中，因此交换结构可看成是“**在路由器中的网络**”。

请注意“转发”和“路由选择”是有区别的。在互联网中，“转发”就是路由器根据转发表把收到的 IP 数据报从路由器合适的端口转发出去。“转发”仅仅涉及一个路由器。但“**路由选择**”则涉及**很多路由器**，路由表则是许多路由器协同工作的结果。这些路由器按照复杂的路由算法，得出整个网络的拓扑变化情况，因而能够动态地改变所选择的路由，并由此构造出整个的路由表。路由表一般仅包含从目的网络到下一跳（用 IP 地址表示）的映射，而转发表是从路由表得出的。转发表必须包含完成转发功能所必需的信息。这就是说，在转发表的每一行必须包含从要到达的目的网络到输出端口和某些 MAC 地址信息（如下一跳的以太网地址）的映射。将转发表和路由表用不同的数据结构实现会带来一些好处，这是因为在转发分组时，转发表的结构应当使查找过程最优化，但路由表则需要对网络拓扑变化的计算最优化。路由表总是用软件实现的，但转发表则可用特殊的硬件来实现。请读者注意，在讨论路由选择的原理时，往往不去区分转发表和路由表的区别，而可以笼统地都使用路由表这一名词。

在图 4-54 中，路由器的输入和输出端口里面都各有三个方框，用方框中的 1、2 和 3 分别代表物理层、数据链路层和网络层的处理模块。物理层进行比特的接收。数据链路层则按照链路层协议接收传送分组的帧。在把帧的首部和尾部剥去后，分组就被送入网络层的处理模块。若接收到的分组是路由器之间交换路由信息的分组（如 RIP 或 OSPF 分组等），则把这种分组送交路由器的路由选择部分中的路由选择处理机。若接收到的是数据分组，则按照分组首部中的目的地址查找转发表，根据得出的结果，分组就经过交换结构到达合适的输出端口。一个路由器的输入端口和输出端口就做在路由器的**线路接口卡**上。

输入端口中的查找和转发功能在路由器的交换功能中是最重要的。为了使交换功能分散化，往往把复制的转发表放在每一个输入端口中（如图 4-54 中的虚线箭头所示）。路由选择处理机负责对各转发表的副本进行更新。这些副本常称为“**影子副本**”(shadow copy)。分散化交换可以避免在路由器中的某一点上出现瓶颈。

以上介绍的查找转发表和转发分组的概念虽然并不复杂，但在具体的实现中还会遇到不少困难。问题就在于路由器必须以很高的速率转发分组。最理想的情况是输入端口的处理速率能够跟上线路把分组传送到路由器的速率。这种速率称为**线速**(line speed 或 wire speed)。可以粗略地估算一下。设线路是 OC-48 链路，即 2.5 Gbit/s。若分组长度为 256 字节，那么线速就应当达到每秒能够处理 100 万以上的分组。现在常用 Mpps（百万分组每秒）为单位来说明一个路由器对收到的分组的处理速率有多高。在路由器的设计中，怎样提高查找转发表的速率是一个十分重要的研究课题。

当一个分组正在查找转发表时，后面又紧跟着从这个输入端口收到另一个分组。这个后到的分组就必须在队列中排队等待，因而产生了一定的时延。图 4-55 给出了在输入端口的队列中排队的分组的示意图。

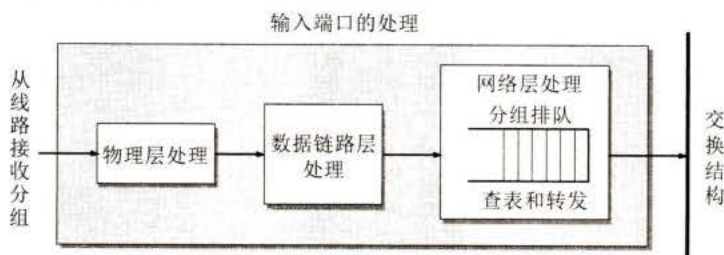


图 4-55 输入端口对线路上收到的分组的处理

我们再来观察在输出端口上的情况（如图 4-56 所示）。输出端口从交换结构接收分组，然后把它们发送到路由器外面的线路上。在网络层的处理模块中设有一个缓存区，实际上它就是一个队列。当交换结构传送过来的分组的速率超过输出链路的发送速率时，来不及发送的分组就必须暂时存放在这个队列中。数据链路层处理模块把分组加上链路层的首部和尾部，交给物理层后发送到外部线路。

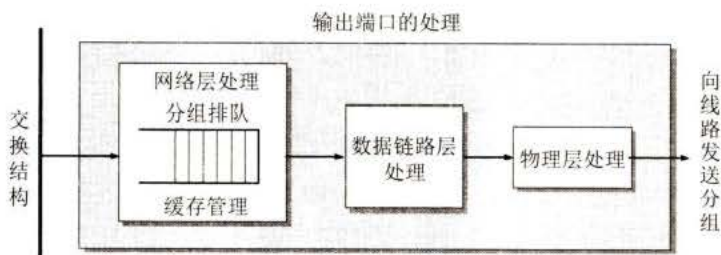


图 4-56 输出端口把交换结构传送过来的分组发送到线路上

从以上的讨论可以看出，分组在路由器的输入端口和输出端口都可能会在队列中排队等候处理。若分组处理的速率赶不上分组进入队列的速率，则队列的存储空间最终必定减少到零，这就使后面再进入队列的分组由于没有存储空间而只能被丢弃。以前我们提到过的分组丢失就是发生在路由器中的输入或输出队列产生溢出的时候。当然，设备或线路故障也可能使分组丢失。

2. 交换结构

交换结构是路由器的关键构件[KURO17]。正是这个交换结构把分组从一个输入端口转移到某个合适的输出端口。实现这样的交换有多种方法，图 4-57 给出了三种常用的交换方法。输入端口是 A、B 和 C，输出端口是 X、Y 和 Z。下面简单介绍它们的特点。

最早使用的路由器就是利用普通的计算机，用计算机的 CPU 作为路由器的路由选择处理机。路由器的输入和输出端口的功能和传统的操作系统中的 I/O 设备一样。当路由器的某个输入端口收到一个分组时，就用中断方式通知路由选择处理机。然后分组就从输入端口复制到存储器中。路由器处理机从分组首部提取目的地址，查找路由表，再将分组复制到合适的输出端口的缓存中。若存储器的带宽（读或写）为每秒 M 个分组，那么路由器的交换速率（即分组从输入端口传送到输出端口的速率）一定小于 $M/2$ 。这是因为存储器对分组的读

和写需要花费的时间是同一个数量级。

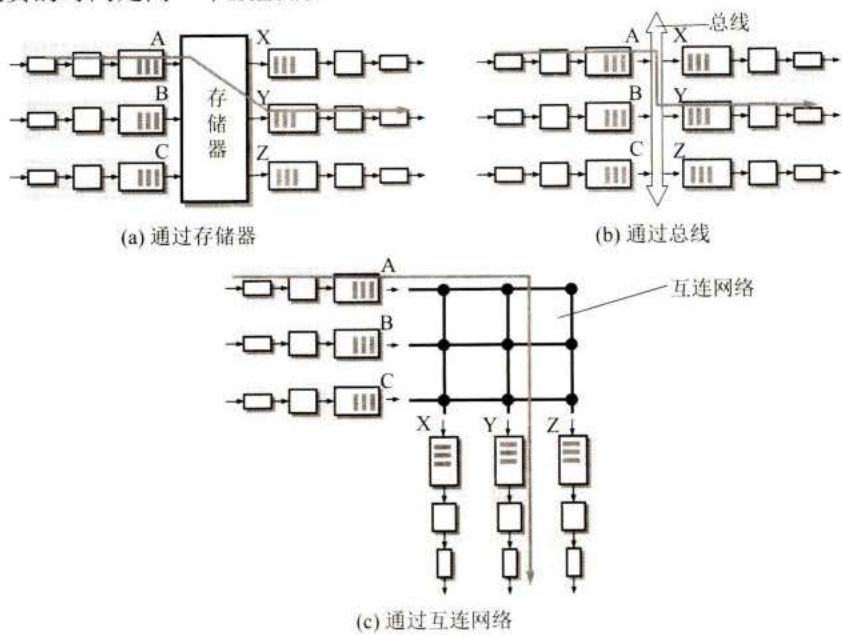


图 4-57 三种常用的交换方法

许多现代的路由器也通过存储器进行交换，图 4-57(a)的示意图表示分组通过存储器进行交换。与早期的路由器的区别就是，目的地址的查找和分组在存储器中的缓存都是在输入端口中进行的。思科(Cisco)公司的 Catalyst 8500 系列交换机（有的公司把路由器也称为交换机）就采用了共享存储器的方法。

图 4-57(b)是通过总线进行交换的示意图。采用这种方式时，数据报从输入端口通过共享总线直接传送到合适的输出端口，而不需要路由选择处理机的干预。但是，由于总线是共享的，因此在同一时间只能有一个分组在总线上传送。当分组到达输入端口时若发现总线忙（因为总线正在传送另一个分组），则被阻塞而不能通过交换结构，并在输入端口排队等待。因为每一个要转发的分组都要通过这一条总线，因此路由器的转发带宽就受总线速率的限制。现代的技术已经可以将总线的带宽提高到能够满足小型企业网的需求。例如，思科公司的 6500 路由器用来交换分组的背板总线速率已达到 32 Gbit/s。

图 4-57(c)画的是通过纵横交换结构(crossbar switch fabric)进行交换。这种交换机构常称为互连网络(interconnection network)，它有 $2N$ 条总线，可以使 N 个输入端口和 N 个输出端口相连接，这取决于相应的交叉节点是使水平总线和垂直总线接通还是断开。当输入端口收到一个分组时，就将它发送到与该输入端口相连的水平总线上。若通向所要转发的输出端口的垂直总线是空闲的，则在这个节点将垂直总线与水平总线接通，然后把该分组转发到这个输出端口。例如，一个分组到达输入端口 A，应转发到输出端口 Y。这时交换结构的控制器就把总线 A 和 Y 的交叉节点闭合，因此分组就从输入端口 A 传送到了输出端口 Y。请注意，如果与此同时还有一个分组要从输入端口 B 转发到输出端口 Z，那么也可同时进行，因为 A → Y 和 B → Z 的转发是使用不同的总线的转发。和前两种交换机制不同，这种纵横交换结构是一种无阻塞的交换结构，其特点是分组可以转发到任何一个输出端口，只要这个输出端口没有被别的分组占用。如果这个输出端口（即对应的垂直总线）已被占用（有另一个分组正

在转发到同一个输出端口), 那么后到达的分组就必须在输入端口排队等待。采用这种交换方式的路由器例子如思科公司的 12000 系列交换路由器, 其互连网络的速率达 60 Gbit/s。

4.7 IP 多播

4.7.1 IP 多播的基本概念

1988 年 Steve Deering 首次在其博士学位论文中提出 IP 多播的概念。1992 年 3 月 IETF 在互联网范围首次试验 IETF 会议声音的多播, 当时有 20 个网点可同时听到会议的声音。IP 多播是需要在互联网上增加更多的智能才能提供的一种服务。现在 IP 多播 (multicast, 以前曾译为组播) 已成为互联网的一个热门课题。这是由于有许多的应用需要由一个源点发送到许多个终点, 即一对多的通信。例如, 实时信息的交付 (如新闻、股市行情等)、软件更新、交互式会议等。随着互联网的用户数目的急剧增加, 以及多媒体通信的开展, 有更多的业务需要多播来支持。

与单播相比, 在一对多的通信中, 多播可大大节约网络资源。图 4-58(a) 是视频服务器用单播方式向 90 台主机传送同样的视频节目。为此, 需要发送 90 个单播, 即同一个视频分组要发送 90 个副本。图 4-58(b) 是视频服务器用多播方式向属于同一个多播组的 90 个成员传送节目。这时, 视频服务器只需把视频分组当作多播数据报来发送, 并且只需发送一次。路由器 R_1 在转发分组时, 需要把收到的分组复制成 3 个副本, 分别向 R_2 、 R_3 和 R_4 各转发 1 个副本。当分组到达目的局域网时, 由于局域网具有硬件多播功能, 因此不需要复制分组, 在局域网上的多播组成员都能收到这个视频分组。

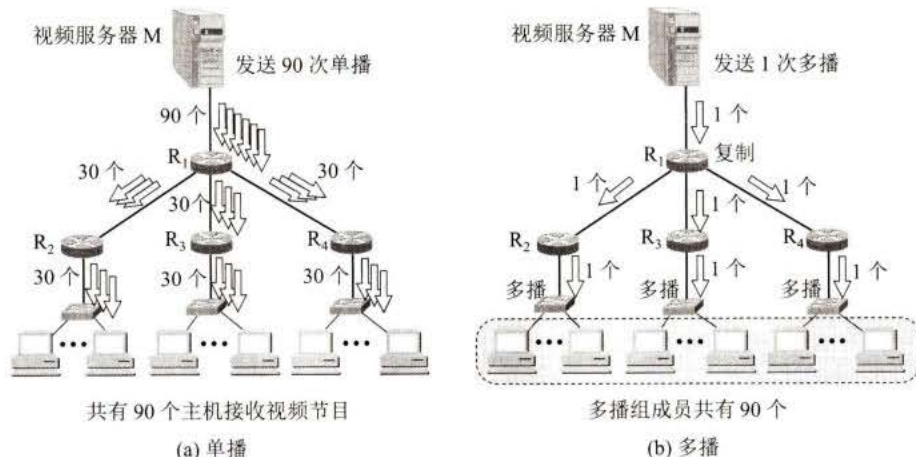


图 4-58 单播与多播的比较

当多播组的主机数很大时 (如成千上万个), 采用多播方式就可明显地减轻网络中各种资源的消耗。在互联网范围的多播要靠路由器来实现, 这些路由器必须增加一些能够识别多播数据报的软件。能够运行多播协议的路由器称为多播路由器 (multicast router)。多播路由器当然也可以转发普通的单播 IP 数据报。

为了适应交互式音频和视频信息的多播, 从 1992 年起, 在互联网上开始试验虚拟的多播主干网 MBONE (Multicast Backbone On the InterNet)。MBONE 可把分组传播给地点分散但属于一个组的许多台主机。现在多播主干网已经有了相当大的规模。

在互联网上进行多播就叫作 **IP 多播**。IP 多播所传送的分组需要使用多播 IP 地址。

我们知道，在互联网中每一台主机必须有一个全球唯一的 IP 地址。如果某台主机现在想接收某个特定多播组的分组，那么怎样才能使这个多播数据报传送到这台主机？

显然，这个多播数据报的目的地址一定不能写入这台主机的 IP 地址。这是因为在同一时间可能有成千上万台主机加入到同一个多播组。多播数据报不可能在其首部写入这样多的主机的 IP 地址。在多播数据报的目的地址写入的是多播组的标识符，然后设法让加入到这个多播组的主机的 IP 地址与多播组的标识符关联起来。

其实多播组的标识符就是 IP 地址中的 D 类地址。D 类 IP 地址的前四位是 1110，因此 D 类地址范围是 224.0.0.0 到 239.255.255.255。我们就用每一个 D 类地址标志一个多播组。这样，D 类地址共可标志 2^{28} 个多播组，也就是说，在同一时间可以允许有超过 2.6 亿的多播组在互联网上运行。多播数据报也是“尽最大努力交付”，不保证一定能够交付多播组内的所有成员。因此，多播数据报和一般的 IP 数据报的区别就是它使用 D 类 IP 地址作为目的地址，并且首部中的协议字段值是 2，表明使用网际组管理协议 IGMP。

显然，多播地址只能用于目的地址，而不能用于源地址。此外，对多播数据报不产生 ICMP 差错报文。因此，若在 PING 命令后面键入多播地址，将永远不会收到响应。

IP 多播可以分为两种。一种是只在本局域网上进行硬件多播，另一种则是在互联网的范围进行多播。前一种虽然比较简单，但很重要，因为现在大部分主机都是通过局域网接入到互联网的。在互联网上进行多播的最后阶段，还是要把多播数据报在局域网上用硬件多播交付多播组的所有成员（如图 4-58(b)所示）。下面就先讨论这种硬件多播。

4.7.2 在局域网上进行硬件多播

互联网号码指派管理局 IANA 拥有的以太网地址块的高 24 位为 00-00-5E，因此 TCP/IP 协议使用的以太网地址块的范围是从 00-00-5E-00-00-00 到 00-00-5E-FF-FF-FF。在第 3 章 3.3.5 节已讲过，以太网 MAC 地址字段中的第 1 字节的最低位为 1 时即为多播地址，这种多播地址数占 IANA 分配到的地址数的一半。但 IANA 只拿出 2^{23} 个地址，即 01-00-5E-00-00-00 到 01-00-5E-7F-FF-FF 的地址作为以太网多播地址。或者说，在 48 位的多播地址中，前 25 位都固定不变，只有后 23 位可用作多播。但 D 类 IP 地址可供分配的有 28 位。这 28 位中只有后 23 位才映射以太网多播地址中的后 23 位，因此是多对一的映射关系（如图 4-59 所示），即 28 位中的前 5 位不能用来构成以太网多播地址。例如，IP 多播地址 224.128.64.32（即 E0-80-40-20）和另一个 IP 多播地址 224.0.64.32（即 E0-00-40-20）转换成以太网的多播地址都是 01-00-5E-00-40-20。因此收到多播数据报的主机，还要在 IP 层利用 IP 数据报首部的 IP 地址进行过滤，把不是本主机要接收的数据报丢弃。

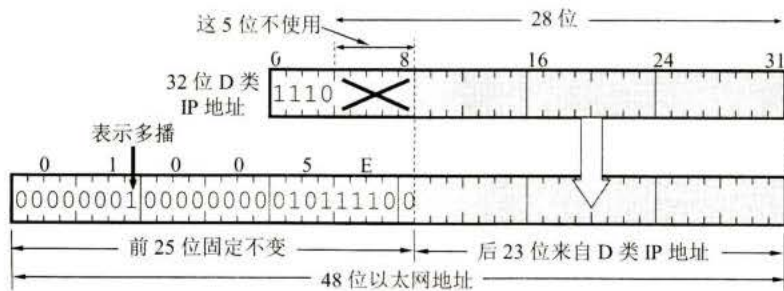


图 4-59 D 类 IP 地址与以太网多播地址的映射关系

下面就讨论进行 IP 多播所需要的协议。

4.7.3 网际组管理协议 IGMP 和多播路由选择协议

1. IP 多播需要两种协议

图 4-60 是在互联网上传送多播数据报的例子。图中标有 IP 地址的四台主机都参加了一个多播组，其组地址是 226.15.37.123。显然，多播数据报应当传送到路由器 R₁、R₂ 和 R₃，而不应当传送到路由器 R₄，因为与 R₄ 连接的局域网上现在没有这个多播组的成员。但这些路由器又怎样知道多播组的成员信息呢？这就要利用一个协议，叫作网际组管理协议 IGMP (Internet Group Management Protocol)。

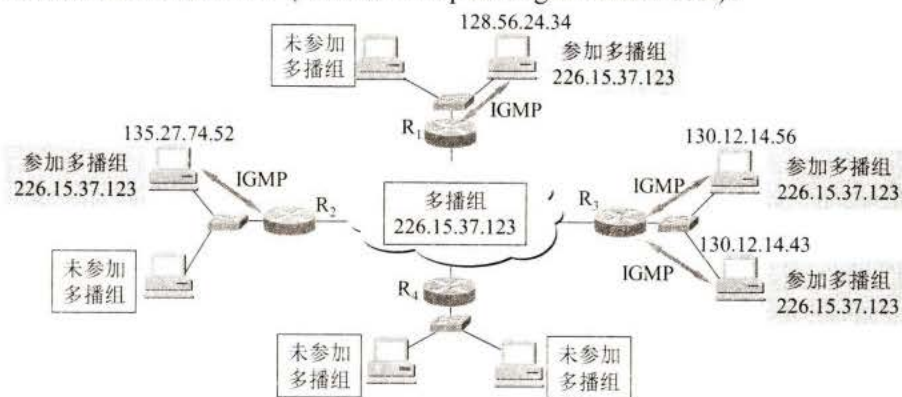


图 4-60 IGMP 使多播路由器知道多播组成员信息

图 4-60 强调了 IGMP 的本地使用范围。请注意，IGMP 并非在互联网范围内对所有多播组成员进行管理的协议。IGMP 不知道 IP 多播组包含的成员数，也不知道这些成员都分布在哪些网络上。IGMP 协议是让连接在本地局域网上的多播路由器知道本局域网上是否有主机（严格讲，是主机上的某个进程）参加或退出了某个多播组。

显然，仅有 IGMP 协议是不能完成多播任务的。连接在局域网上的多播路由器还必须和互联网上的其他多播路由器协同工作，以便把多播数据报用最小代价传送给所有的组成员。这就需要使用多播路由选择协议。

然而多播路由选择协议要比单播路由选择协议复杂得多。我们可以通过一个简单的例子来说明。我们假定图 4-61 中有两个多播组。多播组 M₁ 的成员有主机 A、B 和 C，而多播组 M₂ 的成员有主机 D、E 和 F。这些主机分布在三个网络上 (N₁、N₂ 和 N₃)。

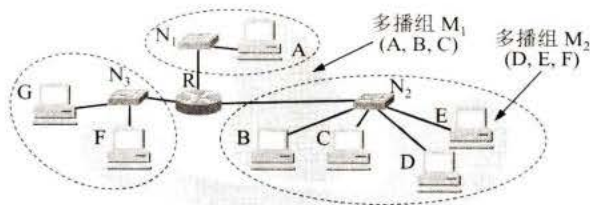


图 4-61 用来说明多播路由选择的例子

路由器 R 不应当向网络 N₃ 转发多播组 M₁ 的分组，因为网络 N₃ 上没有多播组 M₁ 的成员。但是每一台主机可以随时加入或离开一个多播组。例如，如果主机 G 现在加入了多播

组 M_1 ，那么从这时起，路由器 R 就必须也向网络 N_3 转发多播组 M_1 的分组。这就是说，多播转发必须动态地适应多播组成员的变化（这时网络拓扑并未发生变化）。请注意，单播路由选择通常在网络拓扑发生变化时才需要更新路由。

再看一种情况。主机 E 和 F 都是多播组 M_2 的成员。当 E 向 F 发送多播数据报时，路由器 R 把这个多播数据报转发到网络 N_3 。但当 F 向 E 发送多播数据报时，路由器 R 则把多播数据报转发到网络 N_2 。如果路由器 R 收到来自主机 A 的多播数据报（A 不是多播组 M_2 的成员，但也可向多播组发送多播数据报），那么路由器 R 就应当把多播数据报转发到 N_2 和 N_3 。由此可见，多播路由器在转发多播数据报时，不能仅仅根据多播数据报中的目的地址，而是还要考虑这个多播数据报从什么地方来和要到什么地方去。

还有一种情况。主机 G 没有参加任何多播组，但 G 却可向任何多播组发送多播数据报。例如，G 可向多播组 M_1 或 M_2 发送多播数据报。主机 G 所在的局域网上可以没有任何多播组的成员。显然，多播数据报所经过的许多网络，也不一定非要有多播组成员。总之，多播数据报可以由没有加入多播组的主机发出，也可以通过没有组成员接入的网络。

正因为如此，IP 多播就成为比较复杂的问题。下面介绍这两种协议的要点。

2. 网际组管理协议 IGMP

IGMP 已有了三个版本。1989 年公布的 RFC 1112 (IGMPv1) 早已成为了互联网的标准协议 (STD 5)。2002 年 10 月公布的建议标准 IGMPv3 是最新的[RFC 3376]。

和网际控制报文协议 ICMP 相似，IGMP 使用 IP 数据报传递其报文（即 IGMP 报文加上 IP 首部构成 IP 数据报），但它也向 IP 提供服务。因此，我们不把 IGMP 看成是一个单独的协议，而是属于整个网际协议 IP 的一个组成部分。

从概念上讲，IGMP 的工作可分为两个阶段。

第一阶段：当某台主机加入新的多播组时，该主机应向多播组的多播地址发送一个 IGMP 报文，声明自己要成为该组的成员。本地的多播路由器收到 IGMP 报文后，还要利用多播路由选择协议把这种组成员关系转发给互联网上的其他多播路由器。

第二阶段：组成员关系是动态的。本地多播路由器要周期性地探询本地局域网上的主机，以便知道这些主机是否还继续是组的成员。只要有一台主机对某个组响应，那么多播路由器就认为这个组是活跃的。但一个组在经过几次的探询后仍然没有一台主机响应，多播路由器就认为本网络上的主机已经都离开了这个组，因此也就不再把这个组的成员关系转发给其他的多播路由器。

IGMP 设计得很仔细，避免了多播控制信息给网络增加大量的开销。IGMP 采用的一些具体措施如下：

(1) 在主机和多播路由器之间的所有通信都使用 IP 多播。只要有可能，携带 IGMP 报文的数据报都用硬件多播来传送。因此在支持硬件多播的网络上，没有参加 IP 多播的主机不会收到 IGMP 报文。

(2) 多播路由器在探询组成员关系时，只需要对所有的组发送一个请求信息的询问报文，而不需要对每一个组发送一个询问报文（虽然也允许对一个特定组发送询问报文）。默认的询问速率是每 125 秒发送一次（通信量并不太大）。

(3) 当同一个网络上连接有几个多播路由器时，它们能够迅速和有效地选择其中的一个来探询主机的成员关系。因此，网络上多个多播路由器并不会引起 IGMP 通信量的增大。

(4) 在 IGMP 的询问报文中有一个数值 N ，它指明一个最长响应时间（默认值为 10 秒）。当收到询问时，主机在 0 到 N 之间随机选择发送响应所需经过的时延。因此，若一台主机同时参加了几个多播组，则主机对每一个多播组选择不同的随机数。对应于最小时延的响应最先发送。

(5) 同一个组内的每一台主机都要监听响应，只要有本组的其他主机先发送了响应，自己就可以不再发送响应了。这样就抑制了不必要的通信量。

多播路由器并不需要保留组成员关系的准确记录，因为向局域网上的组成员转发数据报是使用硬件多播。多播路由器只需要知道网络上是否至少还有一台主机是本组成员即可。实际上，对询问报文每一个组只需有一台主机发送响应。

如果一台主机上有多个进程都加入了某个多播组，那么这台主机对发给这个多播组的每个多播数据报只接收一个副本，然后给主机中的每一个进程发送一个本地复制的副本。

最后我们还要强调指出，多播数据报的发送者和接收者都不知道（也无法找出）一个多播组的成员有多少，以及这些成员是哪些主机。互联网中的路由器和主机都不知道哪个应用进程将要向哪个多播组发送多播数据报，因为任何应用进程都可以在任何时候向任何一个多播组发送多播数据报，而这个应用进程并不需要加入这个多播组。

IGMP 的报文格式可参阅有关文档[RFC 3376]，这里从略。

3. 多播路由选择协议

虽然在 TCP/IP 中 IP 多播协议已成为建议标准，但多播路由选择协议（用来在多播路由器之间传播路由信息）则尚未标准化。

在多播过程中一个多播组中的成员是动态变化的。例如在收听网上某个广播节目时，随时会有主机加入或离开这个多播组。多播路由选择实际上就是要找出以源主机为根节点的多播转发树。在多播转发树上，每一个多播路由器向树的叶节点方向转发收到的多播数据报，但在多播转发树上的路由器不会收到重复的多播数据报（即多播数据报不应在互联网中兜圈子）。不难看出，对不同的多播组对应于不同的多播转发树。同一个多播组，对不同的源点也会有不同的多播转发树。

已有了多种实用的多播路由选择协议，它们在转发多播数据报时使用了以下的三种方法：

(1) **洪泛与剪除**。这种方法适合于较小的多播组，而所有的组成员接入的局域网也是相邻接的。一开始，路由器转发多播数据报使用洪泛的方法（这就是广播）。为了避免兜圈子，采用了叫作**反向路径广播 RPB (Reverse Path Broadcasting)**的策略。RPB 的要点是：每一个路由器在收到一个多播数据报时，先检查数据报是否是从源点经最短路径传送来的。进行这种检查很容易，只要从本路由器寻找到源点的最短路径上（之所以叫作反向路径，因为在计算最短路径时是把源点当作终点的）的第一个路由器是否就是刚才把多播数据报送来的路由器。若是，就向所有其他方向转发刚才收到的多播数据报（但进入的方向除外），否则就丢弃而不转发。如果本路由器有好几个相邻路由器都处在到源点的最短路径上（也就是说，存在几条同样长度的最短路径），那么只能选择一条最短路径，选择的准则就是看这几条最短路径中的相邻路由器谁的 IP 地址最小。图 4-62 的例子说明了这一概念。

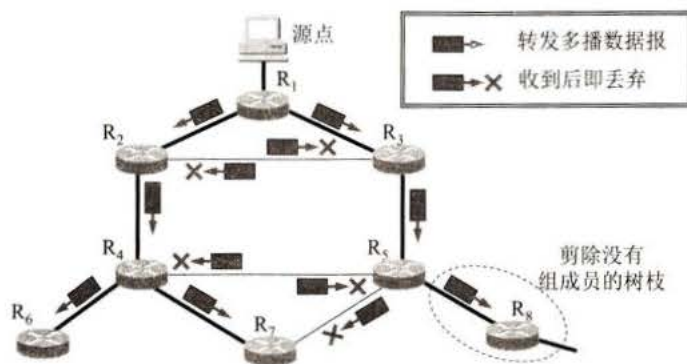


图 4-62 反向路径广播 RPB 和剪除

为简单起见，在图 4-62 中的网络用路由器之间的链路来表示。我们假定各路由器之间的距离都是 1。路由器 R_1 收到源点发来的多播数据报后，向 R_2 和 R_3 转发。 R_2 发现 R_1 就在自己到源点的最短路径上，因此向 R_3 和 R_4 转发收到的数据报。 R_3 发现 R_2 不在自己到源点的最短路径上，因此丢弃 R_2 发来的数据报。其他路由器也这样转发。 R_7 到源点有两条最短路径： $R_7 \rightarrow R_4 \rightarrow R_2 \rightarrow R_1 \rightarrow$ 源点； $R_7 \rightarrow R_5 \rightarrow R_3 \rightarrow R_1 \rightarrow$ 源点。我们再假定 R_4 的 IP 地址比 R_5 的 IP 地址小，所以我们只使用前一条最短路径。因此 R_7 只转发 R_4 传过来的数据报，而丢弃 R_5 传过来的数据报。最后就得出了用来转发多播数据报的多播转发树（图中用粗线表示），以后就按这个多播转发树来转发多播数据报。这样就避免了多播数据报兜圈子，同时每一个路由器也不会接收重复的多播数据报。

如果在多播转发树上的某个路由器发现它的下游树枝（即叶节点方向）已没有该多播组的成员，就应把它和下游的树枝一起剪除。例如，在图 4-62 中虚线椭圆表示剪除的部分。当某个树枝有新增加的组成员时，可以再接入到多播转发树上。

(2) 隧道技术(tunneling)。隧道技术适用于多播组的位置在地理上很分散的情况。例如在图 4-63 中，网 N_1 和网 N_2 都支持多播。现在 N_1 中的主机向 N_2 中的一些主机进行多播。但路由器 R_1 和 R_2 之间的网络并不支持多播，因而 R_1 和 R_2 不能按多播地址转发数据报。为此，路由器 R_1 就对多播数据报进行再次封装，即再加上普通数据报首部，使之成为向单一目的站发送的单播(unicast)数据报，然后通过“隧道”(tunnel)从 R_1 发送到 R_2 。

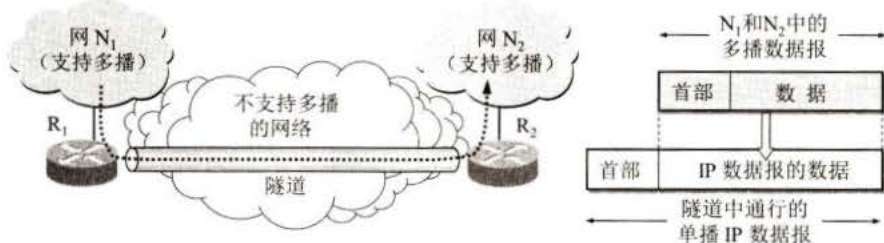


图 4-63 隧道技术在多播中的应用

单播数据报到达路由器 R_2 后，再由路由器 R_2 剥去其首部，使它又恢复成原来的多播数据报，继续向多个目的站转发。这一点和英吉利海峡隧道运送汽车的情况相似。英吉利海峡隧道不允许汽车在隧道中行驶。但是，可以把汽车放置在隧道中行驶的电气火车上来通过隧道。过了隧道后，汽车又可以继续在公路上行驶。这种使用隧道技术传送数据报又叫作 IP

中的 IP (IP-in-IP)。

(3) **基于核心的发现技术**。这种方法对于多播组的大小在较大范围内变化时都适合。这种方法是对每一个多播组 G 指定一个**核心(core)路由器**，给出它的 IP 单播地址。核心路由器按照前面讲过的方法创建出对应于多播组 G 的转发树。如果有一个路由器 R₁ 向这个核心路由器发送数据报，那么它在途中经过的每一个路由器都要检查其内容。当数据报到达参加了多播组 G 的路由器 R₂ 时，R₂ 就处理这个数据报。如果 R₁ 发出的是一个多播数据报，其目的地址是 G 的组地址，R₂ 就向多播组 G 的成员转发这个多播数据报。如果 R₁ 发出的数据报是一个请求加入多播组 G 的数据报，R₂ 就把这个信息加到它的路由中，并用隧道技术向 R₁ 转发每一个多播数据报的一个副本。这样，参加到多播组 G 的路由器就从核心向外增多了，扩大了多播转发树的覆盖范围。

目前还没有在整个互联网范围使用的多播路由选择协议。下面是一些建议使用的多播路由选择协议。

距离向量多播路由选择协议 DVMRP (Distance Vector Multicast Routing Protocol)是在互联网上使用的第一个多播路由选择协议[RFC 1075]。由于在 UNIX 系统中实现 RIP 的程序叫作 routed，所以在 routed 的前面加表示多播的字母 m，叫作 mroute，它使用 DVMRP 在路由器之间传播路由信息。

基于核心的转发树 CBT (Core Based Tree) [RFC 2189, 2201]。这个协议使用核心路由器作为转发树的根节点。一个大的自治系统 AS 可划分为几个区域，每一个区域选择一个核心路由器（也叫作中心路由器 center router，或汇聚点路由器 rendezvous router）。

开放最短通路优先的多播扩展 MOSPF (Multicast extensions to OSPF) [RFC 1585]。这个协议是单播路由选择协议 OSPF 的扩充，使用于一个机构内。MOSPF 使用多播链路状态路由选择创建出基于源点的多播转发树。

协议无关多播-稀疏方式 PIM-SM (Protocol Independent Multicast-Sparse Mode) [RFC 7761, STD83]。这是唯一成为互联网标准的一个协议，它使用和 CBT 同样的方法构成多播转发树。采用“协议无关”这个名词是强调：虽然在建立多播转发树时是使用单播数据报来和远程路由器联系的，但这并不要求使用特定的单播路由选择协议。这个协议适用于组成员的分布非常分散的情况。

协议无关多播-密集方式 PIM-DM (Protocol Independent Multicast-Dense Mode) [RFC 3973]。这个协议适用于组成员的分布非常集中的情况，例如组成员都在一个机构之内。PIM-DM 不使用核心路由器，而是使用洪泛方式转发数据报。

4.8 虚拟专用网 VPN 和网络地址转换 NAT

4.8.1 虚拟专用网 VPN

由于 IP 地址的紧缺，一个机构能够申请到的 IP 地址数往往远小于本机构所拥有的主机数。考虑到互联网并不很安全，一个机构内也并不需要把所有的主机接入到外部的互联网。实际上，在许多情况下，很多主机主要还是和本机构内的其他主机进行通信（例如，在大型商场或宾馆中，有很多用于营业和管理的计算机。显然这些计算机并不都需要和互联网相连）。假定在一个机构内部的计算机通信也是采用 TCP/IP 协议，那么从原则上讲，对于这些

仅在机构内部使用的计算机就可以由本机构自行分配其 IP 地址。这就是说,让这些计算机使用仅在本机构有效的 IP 地址(这种地址称为**本地地址**),而不需要向互联网的管理机构申请全球唯一的 IP 地址(这种地址称为**全球地址**)。这样就可以大大节约宝贵的全球 IP 地址资源。

但是,如果任意选择一些 IP 地址作为本机构内部使用的本地地址,那么在某种情况下可能会引起一些麻烦。例如,有时机构内部的某台主机需要和互联网连接,那么这种仅在内部使用的本地地址就有可能和互联网中某个 IP 地址重合,这样就会出现地址的二义性问题。

为了解决这一问题,RFC 1918 指明了一些**专用地址(private address)**。这些地址只能用于一个机构的内部通信,而不能用于和互联网上的主机通信。换言之,专用地址只能用作本地地址而不能用作全球地址。在互联网中的所有路由器,对目的地址是专用地址的数据报一律不进行转发。2013 年 4 月,RFC 6890 全面地给出了所有特殊用途的 IPv4 和 IPv6 地址,但三个 IPv4 专用地址块的指派并无变化,即

- (1) 10.0.0.0/8,即从 10.0.0.0 到 10.255.255.255。
- (2) 172.16.0.0/12,即从 172.16.0.0 到 172.31.255.255。
- (3) 192.168.0.0/16,即从 192.168.0.0 到 192.168.255.255。

上面的三个地址块分别相当于原来的一个 A 类网络、16 个连续的 B 类网络和 256 个连续的 C 类网络。A 类地址本来早已用完了,而上面的地址 10.0.0.0 本来是分配给 ARPANET 的。由于 ARPANET 已经关闭停止运行了,因此这个地址就用作专用地址。

采用这样的专用 IP 地址的互连网络称为**专用互联网**或**本地互联网**,或更简单些,就叫作**专用网**。显然,全世界可能有很多的专用互连网络具有相同的 IP 地址,但这并不会引起麻烦,因为这些专用地址仅在本机构内部使用。专用 IP 地址也叫作**可重用地址(reusable address)**。

有时一个很大的机构的许多部门分布的范围很广(例如,在世界各地),这些部门经常要互相交换信息。这可以有两种方法。(1) 租用电信公司的通信线路为本机构专用。这种方法虽然简单方便,但线路的租金太高,一般难于承受。(2) 利用公用的互联网作为本机构各专用网之间的通信载体,这样的专用网又称为**虚拟专用网 VPN (Virtual Private Network)**。

这里之所以称为“专用网”是因为这种网络是为本机构的主机用于机构内部的通信,而不是用于和网络外非本机构的主机通信。如果专用网不同网点之间的通信必须经过公用的互联网,但又有保密的要求,那么**所有通过互联网传送的数据都必须加密**。加密需要采用的协议将在 7.6.1 节讨论。“虚拟”表示“好像是”,但实际上并不是,因为现在并没有真正使用通信专线,而 VPN 只是在效果上和真正的专用网一样。一个机构要构建自己的 VPN 就必须为它的每一个场所购买专门的硬件和软件,并进行配置,使每一个场所的 VPN 系统都知道其他场所的地址。

图 4-64 以两个场所为例说明如何使用 IP 隧道技术实现虚拟专用网。

假定某个机构在两个相隔较远的场所建立了专用网 A 和 B,其网络地址分别为**专用地址 10.1.0.0**和**10.2.0.0**。现在这两个场所需要通过公用的互联网构成一个 VPN。

显然,每一个场所至少要有有一个路由器具有合法的全球 IP 地址,如图 4-64(a)中的路由器 R_1 和 R_2 。这两个路由器和互联网的接口地址必须是合法的全球 IP 地址。路由器 R_1 和 R_2 在专用网内部网络的接口地址则是专用网的本地地址。

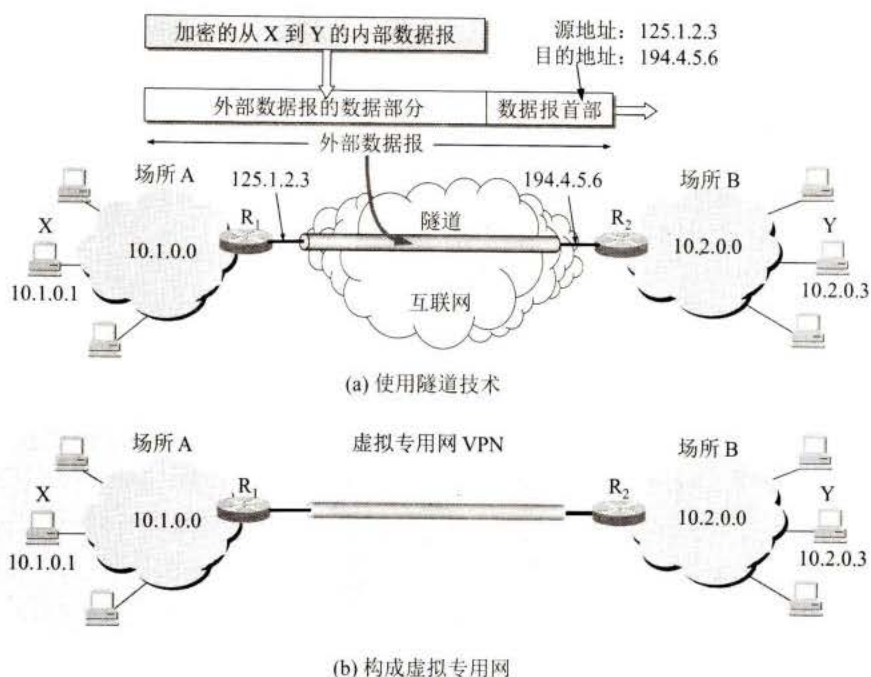


图 4-64 用隧道技术实现虚拟专用网

在每一个场所 A 或 B 内部的通信量都不经过互联网。但如果场所 A 的主机 X 要和另一个场所 B 的主机 Y 通信，那么就必须要经过路由器 R₁ 和 R₂。主机 X 向主机 Y 发送的 IP 数据报的源地址是 10.1.0.1，而目的地址是 10.2.0.3。这个数据报先作为本机构的内部数据报从 X 发送到与互联网连接的路由器 R₁。路由器 R₁ 收到内部数据报后，发现其目的网络必须通过互联网才能到达，就把整个的内部数据报进行加密（这样就保证了内部数据报的安全），然后重新加上数据报的首部，封装成为在互联网上发送的外部数据报，其源地址是路由器 R₁ 的全球地址 125.1.2.3，而目的地址是路由器 R₂ 的全球地址 194.4.5.6。路由器 R₂ 收到数据报后将其数据部分取出进行解密，恢复出原来的内部数据报（目的地址是 10.2.0.3），交付主机 Y。可见，虽然 X 向 Y 发送的数据报通过了公用的互联网，但在效果上就好像是在本部门的专用网上传送一样。如果主机 Y 要向 X 发送数据报，那么所进行的步骤也是类似的。

请注意，数据报从 R₁ 传送到 R₂ 可能要经过互联网中的很多个网络和路由器。但从逻辑上看，在 R₁ 到 R₂ 之间好像是一条直通的点对点链路，图 4-64(a) 中的“隧道”就是这个意思。

如图 4-64(b) 所示的、由场所 A 和 B 的内部网络所构成的虚拟专用网 VPN 又称为内联网 (intranet 或 intranet VPN，即内联网 VPN)，表示场所 A 和 B 都属于同一个机构。

有时一个机构的 VPN 需要有某些外部机构（通常就是合作伙伴）参加进来。这样的 VPN 就称为外联网 (extranet 或 extranet VPN，即外联网 VPN)。

请注意，内联网和外联网都采用了互联网技术，即都是基于 TCP/IP 协议的。

还有一种类型的 VPN，就是远程接入 VPN (remote access VPN)。我们知道，有的公司可能并没有分布在不同场所的部门，但却有很多流动员工在外地工作。公司需要和他们保持联系，有时还可能一起开电话会议或视频会议。远程接入 VPN 可以满足这种需求。在外地工作的员工通过拨号接入互联网，而驻留在员工个人电脑中的 VPN 软件可以在员工的个人电脑和公司的主机之间建立 VPN 隧道，因而外地员工与公司通信的内容也是保密的，员工

们感到好像就是使用公司内部的本地网络。

关于 VPN 隧道的保密问题，在后面的 7.5.1 节还有进一步的论述。

4.8.2 网络地址转换

下面讨论另一种情况，就是在专用网内部的一些主机本来已经分配到了本地 IP 地址（即仅在本专用网内使用的专用地址），但现在又想和互联网上的主机通信（并不需要加密），那么应当采取什么措施呢？

最简单的办法就是设法再申请一些全球 IP 地址。但这在很多情况下是不容易做到的。目前使用得最多的方法是采用网络地址转换。

网络地址转换 NAT (Network Address Translation)方法是在 1994 年提出的。这种方法需要在专用网连接到互联网的路由器上安装 NAT 软件。装有 NAT 软件的路由器叫作 NAT 路由器，它至少有一个有效的外部全球 IP 地址。这样，所有使用本地地址的主机在和外界通信时，都要在 NAT 路由器上将其本地地址转换成全球 IP 地址，才能和互联网连接。

图 4-65 给出了 NAT 路由器的工作原理。在图中，专用网 192.168.0.0 内所有主机的 IP 地址都是本地 IP 地址 192.168.x.x。NAT 路由器至少要有一个全球 IP 地址，才能和互联网相连。在本例中，NAT 路由器的全球 IP 地址是 172.38.1.5（当然，NAT 路由器可以有多个全球 IP 地址）。

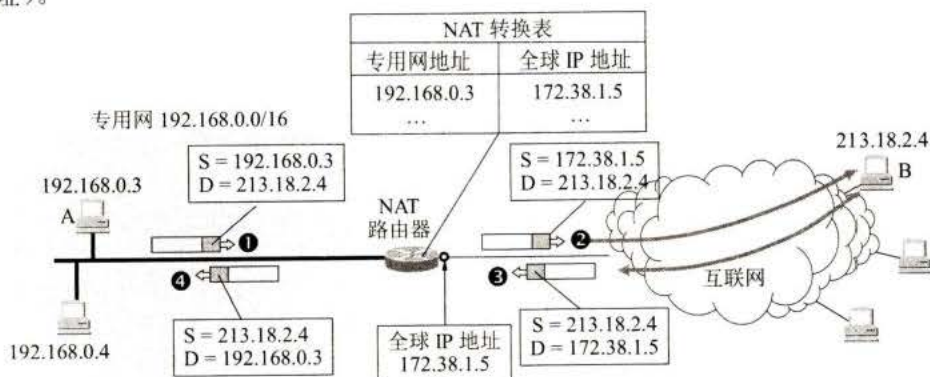


图 4-65 NAT 路由器的工作原理

NAT 路由器收到从专用网内部的主机 A 发往互联网上主机 B 的 IP 数据报①：源地址 S = 192.168.0.3，而目的地址 D = 213.18.2.4。NAT 路由器通过内部的 NAT 转换表，把专用网的 IP 地址 192.168.0.3，转换为全球 IP 地址 172.38.1.5 后，改写到数据报的首部中作为新的源地址，然后把新的数据报②转发出去。主机 B 收到 IP 数据报②后，发回应答③，B 发送的 IP 数据报的源地址就是自己的地址：S = 213.18.2.4，目的地址就是刚才收到的数据报的源地址，因此现在 D = 172.38.1.5。请注意，B 并不知道 A 的专用地址 192.168.0.3。实际上，即使知道了，也不能使用，因为互联网上的路由器不能转发目的地址是任何专用地址的 IP 数据报。当 NAT 路由器收到 B 发来的 IP 数据报③时，还要进行一次 IP 地址的转换。通过 NAT 转换表，把收到的 IP 数据报使用的目的地址 D = 172.38.1.5 转换为专用网内部的目的地址 D = 192.168.0.3（即主机 A 真正的本地 IP 地址），变成了数据报④，然后发送到 A。

由此可见，当 NAT 路由器具有 n 个全球 IP 地址时，专用网内最多可以同时有 n 台主机接入到互联网。这样就可以使专用网内较多数量的主机，轮流使用 NAT 路由器有限数量的

全球 IP 地址。

显然，通过 NAT 路由器的通信必须由专用网内的主机发起。设想如果有专用网上外面的主机要发起通信，当 IP 数据报到达 NAT 路由器时，NAT 路由器就不知道应当把目的 IP 地址转换成专用网内的哪一个本地 IP 地址。这就表明，专用网内部的主机不能直接充当服务器用。

为了更加有效地利用 NAT 路由器上的全球 IP 地址，现在常用的 NAT 转换表把运输层的端口号也利用上。这样，就可以使多个拥有本地地址的主机，共用 NAT 路由器上的一个全球 IP 地址，因而可以同时和互联网上的不同主机进行通信[COME06]。

由于运输层的端口号将在下一章 5.1.3 节讨论，因此，建议在学完运输层的有关内容后，再学习下面的内容。但从系统性方面考虑，把下面的这部分内容放在本章中介绍较为合适。

使用端口号的 NAT 也叫作**网络地址与端口号转换** NAPT (Network Address and Port Translation)，而不使用端口号的 NAT 就叫作传统的 NAT (traditional NAT)。但在许多文献中并没有这样区分，而是不加区分地都使用 NAT 这个更加简洁的缩写词。表 4-9 说明了 NAPT 的地址转换机制。

表 4-9 NAPT 地址转换表举例

方向	字段	原先的 IP 地址和端口号	转换后的 IP 地址和端口号
从专用网发往互联网	源 IP 地址:TCP 源端口	192.168.0.3:30000	172.38.1.5:40001
从专用网发往互联网	源 IP 地址:TCP 源端口	192.168.0.4:30000	172.38.1.5:40002
从互联网发往专用网	目的 IP 地址:TCP 目的端口	172.38.1.5:40001	192.168.0.3:30000
从互联网发往专用网	目的 IP 地址:TCP 目的端口	172.38.1.5:40002	192.168.0.4:30000

从表 4-9 可以看出，在专用网内主机 192.168.0.3 向互联网发送 IP 数据报，其 TCP 端口号选择为 30000。NAPT 把源 IP 地址和 TCP 端口号都进行转换（如果使用 UDP，则对 UDP 的端口号进行转换。原理是一样的）。另一台主机 192.168.0.4 也选择了同样的 TCP 端口号 30000。这纯属巧合（端口号仅在本主机中才有意义）。现在 NAPT 把专用网内不同的源 IP 地址都转换为同样的全球 IP 地址。但对源主机所采用的 TCP 端口号（不管相同或不同），则转换为不同的新的端口号。因此，当 NAPT 路由器收到从互联网发来的应答时，就可以从 IP 数据报的数据部分找出运输层的端口号，然后根据不同的目的端口号，从 NAPT 转换表中找到正确的目的主机。

应当指出，从层次的角度看，NAPT 的机制有些特殊。普通路由器在转发 IP 数据报时，对于源 IP 地址或目的 IP 地址都是不改变的。但 NAT 路由器在转发 IP 数据报时，一定要更换其 IP 地址（转换源 IP 地址或目的 IP 地址）。其次，普通路由器在转发分组时是工作在网络层的。但 NAPT 路由器还要查看和转换运输层的端口号，而这本来应当属于运输层的范畴。也正因为这样，NAPT 曾遭受了一些人的批评，认为 NAPT 的操作没有严格按照层次的关系。但不管怎样，NAT（包括 NAPT）已成为互联网的一个重要构件[RFC 3022]。

4.9 多协议标签交换 MPLS

IETF 于 1997 年成立了 MPLS 工作组，为的是开发出一种新的协议。这种新的协议就是多协议标签交换 MPLS (MultiProtocol Label Switching)。“多协议”表示在 MPLS 的上层可以